Knowledge Graphs for Agentic Software Engineering: A Systematic Review

ANONYMOUS AUTHOR(S)

Knowledge graphs make relationships among software artifacts available to engineering agents, but the effect of graph access depends on how it changes a decision. We systematically review 104 reports spanning agent workflows and the graph mechanisms that can support them, recording workflow control separately from graph use. A Source–Intervention–Validation–Update framework connects five intervention roles to software tasks, acceptance criteria, and maintenance obligations. Our synthesis distinguishes three mechanisms: acquiring missing facts, presenting relationships among known candidates, and deciding whether to seek further evidence require different interventions; graph-supported validity and engineering acceptance can diverge because they assess different targets; and refreshing a graph does not by itself establish the validity of cached context or derived artifacts. Evaluation frequently combines changes to graph access, context, and workflow, limiting attribution of observed gains. Exploratory repository-task comparisons likewise show no uniform quality gain from graph augmentation in the examined configurations. These findings motivate research on selecting useful graph operations, connecting feedback to task acceptance, and coordinating knowledge maintenance across software changes.

Additional Key Words and Phrases: knowledge graphs, agentic software engineering, code graphs, graph retrieval, repository understanding, software evolution

1 Introduction

A software change rarely ends where it begins. An API edit can affect callers, tests, configuration, and explanations retained from an earlier task. Understanding relationships among these artifacts helps developers interpret behavior and coordinate changes [93, 105]. LLM-based software agents must also identify relevant relationships, act on them, and interpret feedback as they explore repositories and modify software [54, 117]. The needed information is often distributed: an implementation decision can depend on an API contract documented outside the repository, and an earlier change can explain a failure observed during testing.

Knowledge graphs represent software artifacts as entities with explicit relationships [32]. These relationships support queries that connect a task to relevant code and its dependencies. For example, CodexGraph provides graph-based access to repositories, and RepoHyper uses semantic graphs to select context for code completion [60, 83]. Development-artifact links guide repair in KGCompass; RepoDoc follows code and concept dependencies to update documentation [111, 115]. Integration with LLMs makes these operations available within reasoning and retrieval workflows [17, 36]. The engineering task shapes how this knowledge is used. During implementation, an agent may need a resolved API relationship to choose an edit. Testing supplies observations about the resulting behavior, and subsequent changes create new demands for keeping retrieved context and derived documents current. Studying KG support across these activities therefore requires following a relationship into the decision it informs and the outcome being assessed. The sources and construction methods matter because they determine which relationships are available; the workflow determines how those relationships enter action and feedback.

Existing surveys organize several parts of this problem. Software-KG reviews cover knowledge construction and applications [3, 105]; the repository-intelligence review compares representations, repository tasks, and evaluation strategies [93]. GraphRAG surveys examine indexing and retrieval,

2026. ACM 1557-7341/2026/10-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

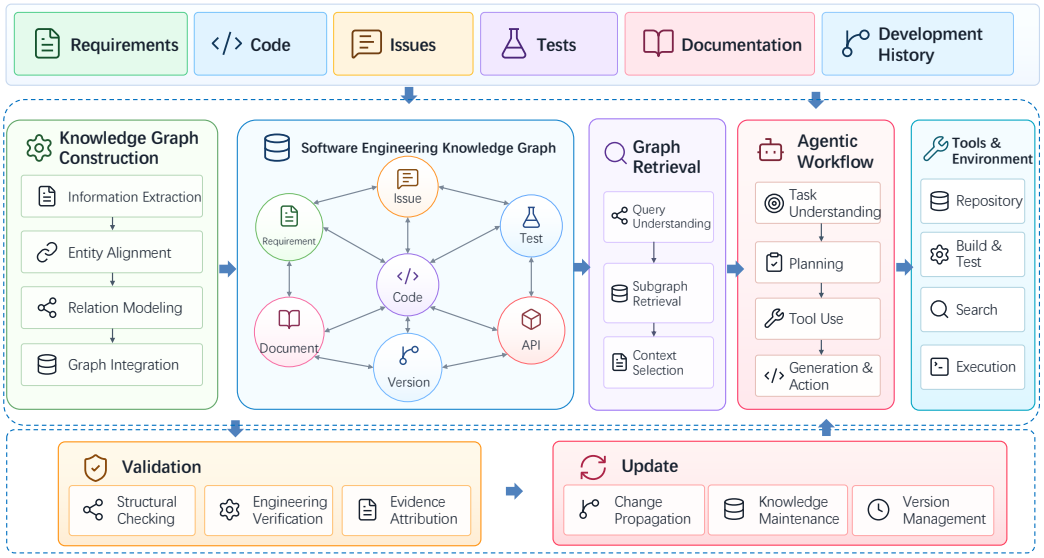


Fig. 1. Overview of knowledge graph support for agentic software engineering. Graph construction connects software artifacts, retrieval supplies context for agent workflows, and validation and update provide feedback as software changes.

and Zhu et al.'s taxonomy also covers workflow control and temporal retrieval [80, 132]. Software-agent and graph–agent surveys organize planning, tool use, memory, and coordination, including updates to retained knowledge [13, 54]. Deng et al. distinguish final-output correctness from intermediate-trajectory validity in a data-lifecycle account of agentic retrieval [20].

Our comparison follows a software relationship through the operation it changes, the acceptance evidence for that operation, and the objects that become invalid after an edit. For example, a dependency can select a file for inspection, constrain an API migration, or identify documentation to regenerate; these uses require different observations and checks. This unit of analysis connects the existing taxonomies to configuration-level evidence about engineering decisions. We review 104 reports covering agent workflows and relevant graph mechanisms. Fixed pipelines, developer-led methods, and graph services remain in scope because they establish mechanisms available for integration; their results are not treated as demonstrations of autonomous agent control. Section 4.7 compares the disclosed control modes.

We organize the literature through *Source–Intervention–Validation–Update*, illustrated in Figure 1. This framework follows graph knowledge from its sources to the engineering operation it influences, the evidence used to assess the outcome, and the maintenance required as software changes. Five intervention roles describe how knowledge participates in a method: evidence access, constraints and planning, training supervision, verification, and persistent state. Recording agent control alongside these roles connects a graph operation to the component that invokes it and interprets its result. The synthesis relates the value of graph support to the evidence available at a decision point. Focused fact retrieval supplies missing context; paths among localized candidates inform a subsequent choice [72, 115]. Acceptance criteria give feedback its engineering meaning, with issue-specific tests and expert requirements review supporting different forms of revision [88, 127]. Knowledge retained across tasks introduces maintenance work that follows the dependencies of the

stored context, tests, or documents [75, 111, 126]. We assess these findings through the comparisons reported in the literature and develop their implications for adaptive access and continued reuse. Complementary repository-level experiments add observations about the quality and efficiency of graph-augmented agents. Our contributions are as follows:

- We systematically examine 104 reports on graph-supported software engineering and agent integration. We characterize graph construction, access, and maintenance through five intervention roles, while separately recording who selects subsequent operations after feedback.
- We synthesize cross-study contrasts through Source–Intervention–Validation–Update. The findings distinguish fact acquisition from relational presentation and continued exploration, graph-supported validity from engineering acceptance, and graph refresh from maintenance of dependent knowledge.
- We critically assess evaluation designs and add exploratory repository-task comparisons. We identify what reported results establish about graph interventions, where context and control effects remain entangled, and which maintenance costs require longitudinal evaluation.

2 Background and Scope

2.1 Graph knowledge in software engineering

We use *explicit graph knowledge* to denote an identifiable set of entities and relationships that the method constructs or queries during a software engineering task. We represent it as

$$G_t = (V_t, E_t, \tau_V, \tau_E, A_t, P_t),$$

where entities and edges belong to a repository or environment state at time t; τ_V and τ_E denote their types; A_t contains attributes; and P_t records provenance. We use these components as comparison dimensions; version and provenance support are recorded separately for each system.

This definition includes typed symbol graphs, repository artifact graphs, API and dependency graphs, and domain knowledge linked to software tasks. It does not require RDF or a particular database; knowledge graphs admit different data models, including RDF and property graphs [32]. An abstract syntax tree represents syntactic structure, whereas a program dependence graph makes data and control dependencies explicit [28]. A code property graph combines syntax, control-flow, and program-dependence representations [114]. A property graph, in contrast, is a data model rather than a specific program analysis. These terms therefore identify different representational choices. Similarly, an embedding is not by itself evidence that an explicit graph participates in the downstream method.

The distinction between structural and inferred relationships is consequential for action. A containment edge recovered from a declaration, a dependency obtained from package metadata, a learned similarity link, and an LLM-generated causal explanation have different evidential status. DepsRAG imports structured dependencies, whereas RANGER adds generated semantic descriptions to parsed structure [7, 91]. Preserving those origins lets an agent select the appropriate check before relying on a relation.

2.2 Agentic software engineering

Agentic software engineering uses LLM-based agents to carry out engineering tasks by interacting with software artifacts, development tools, and people [54]. A task may specify a desired feature, describe a failure, request an explanation, or ask for a change to an existing system. The agent interprets this goal in the context of the project, gathers information, and selects operations that move the work forward. Its output can include source changes, tests, design documents,

configuration updates, or an explanation supported by repository evidence. The scope of its responsibility depends on the task, the tools available, and the decisions reserved for a developer.

An agent combines a language model with mechanisms for planning, tool use, and retaining context. Planning decomposes a goal into steps and dependencies, while tool use connects those steps to operations such as searching files, inspecting symbols, editing artifacts, and executing programs. Working context holds the current task, relevant project information, and observations from earlier actions; longer-lived memory can preserve information across tasks. These components support an iterative process: an observation can reveal missing information, invalidate an assumption, or suggest a different next step. Reasoning and action are therefore interleaved, with the environment providing information that the model cannot obtain from its initial prompt alone [117, 118].

This process applies across the software lifecycle. During requirements and design work, agents can help clarify requested behavior, organize specifications, and examine implementation alternatives. During development, they can generate code, adapt existing components, and coordinate changes across files. Testing and debugging involve constructing inputs, examining failures, locating relevant code, and checking candidate repairs. Maintenance and operations extend the same interaction pattern to dependency changes, documentation, configuration, and runtime diagnosis [54]. The relevant artifacts and acceptance criteria differ across these activities: a useful design explanation, a reproducible failure, and a passing regression test answer different engineering questions. Work can be organized around a single agent or distributed among agents with different responsibilities, such as implementation, testing, and review. Collaboration requires exchanging artifacts and feedback, deciding which component acts next, and retaining a consistent view of the task. Developers may provide missing requirements, resolve competing choices, review proposed changes, or decide when the work is ready to accept. Both the allocation of responsibilities and the amount of human involvement can vary within one workflow [54].

The development environment makes this work observable through file contents, search results, compiler diagnostics, test outcomes, and execution logs. Interfaces determine which operations an agent can perform and how their results enter its context [117]. Much of the information needed for a decision is distributed across artifacts: a requirement constrains an implementation, a caller depends on an interface, and a test checks behavior under a particular configuration. Knowledge graphs can make these relationships explicit and available for retrieval or reasoning. They provide a way to organize the project knowledge used during the interaction, alongside the textual descriptions and executable feedback through which the task is understood and checked.

2.3 Example: resolving a browser compatibility issue

OpenClaw issue #131 reports that its browser control interface fails to connect when accessed over HTTP through a LAN or Tailscale IP address.[1] The browser reports that `crypto.randomUUID` is not a function: this API is unavailable in the reported non-secure context. The expected behavior is for the interface to work in that deployment setting. This is a concrete repository task involving an environment constraint, shared functionality, and multiple callers.

The corresponding pull request #133, introduces a shared `generateUUID()` helper and replaces direct calls in both `gateway.ts`, which creates request identifiers, and `controllers/chat.ts`, which creates chat run identifiers. The helper selects an available generation mechanism, and the added tests cover the native API, a `getRandomValues` fallback, and an environment without a crypto object. The repair therefore spans two callers, a new utility, and its tests.

[1] Issue: https://github.com/openclaw/openclaw/issues/131; corresponding fix: https://github.com/openclaw/openclaw/pull/133.

An agent addressing this issue would need to connect the reported browser error to the affected code and determine whether the same assumption appears elsewhere. It could inspect the documented deployment mode, locate the API calls, introduce a common helper, update its callers, and check the resulting behavior. The issue and patch establish the task and the recorded repair; this sequence illustrates how an agent could approach it, rather than reporting an observed execution trace. A knowledge graph could connect the issue to the deployment documentation, the affected API calls to their containing functions, and the new helper to its callers and tests. These relationships would support finding the second caller and identifying checks for the shared change. Validation would still require exercising the supported API and fallback paths, with an integration check for the reported HTTP setting. After the edit, a retained code graph would need to reflect the new imports and calls; cached explanations of the old implementation could also require refreshing. The example thus connects knowledge sources, an engineering decision, executable checks, and updates to retained knowledge within one concrete task.

3 Approach

3.1 Research questions

We organize the review around three research questions.

RQ1: How is graph knowledge constructed, represented, accessed, and maintained across the agent workflow? We examine knowledge sources, graph construction and identity, intervention roles, access mechanisms, and update policies, distinguishing graph capabilities from the controller that invokes them.

RQ2: Which software engineering activities does graph knowledge support, and under what task conditions is it useful? We connect graph interventions to the relationships, decisions, and acceptance requirements involved in each activity.

RQ3: How is graph support evaluated, and what do the reported results establish about its benefits, limitations, and costs? We examine evaluation designs and trace evidence from graph and context quality through agent decisions to engineering outcomes and lifecycle cost.

Building on RQ1–RQ3, Section 7 develops three findings through cross-study comparisons and identifies open challenges and research opportunities.

3.2 Literature search and selection

Search strategy. Following PRISMA reporting terminology [74], we distinguish records, reports, and studies. The initial database search on 13 August 2026 covered arXiv, IEEE Xplore, ACM Digital Library, Web of Science, and SpringerLink, for publications from 2023 to that date. The subsequent citation search ended on 30 September 2026. Two query families combined graph terms with agent/LLM or software-task terms.

Database search. From 9,476 query-hit occurrences, platform and pre-import screening retained 888 records. Recorded restrictions covered relevance, language and subject category. Removing 152 duplicates left 736; title/abstract screening retained 199. All 199 full texts were assessed; 34 exclusions left 165 reports. One version consolidation and three source reassignments yielded 161 database-attributed studies.

Initial citation searching. Five forward/backward rounds recorded 5,248 discoveries and 3,527 unique candidates [109]. Of 242 completed eligibility assessments, 81 exclusions left 161 provisional inclusions. Final attribution assigned 71 studies to this route, yielding a historical collection of 232.

KG relevance and additional searching. The 232 historical entries formed an intermediate collection of graph-related methods. Subsequent keyword-guided manual reading refined KG relevance, excluding 144 whose main graph structure was insufficiently related to KGs and retaining

88 reports as seeds. KG relevance follows the explicit-knowledge boundary in Section 2.1: identifiable entities and relationships must be constructed or queried for the task, rather than a graph term merely naming a model component. Typed code and artifact graphs remain eligible without requiring RDF. This refinement preceded control classification; fixed workflows remain within the analysis scope. A second forward/backward search ended on 30 September 2026. Its 3,992 discovery occurrences yielded 24 retained records after preliminary filtering and manual review; cross-direction merging left 23 candidates. Removing older versions left 20 for final full-text assessment; all were obtained. Four scope exclusions left 16 additional reports, giving 104 (88 + 16).

At each manual screening stage, two reviewers independently read and assessed the records or reports, compared their decisions, and resolved differences through discussion to determine the final selection. Report-level records retain the screening decisions and their rationale, making each report traceable through selection. One report provides a domain-boundary comparison [30]; the other 103 concern software or adjacent operational tasks. Figure 2 summarizes report years and hosting platforms. Hosting identifies the inspected version; evidence is appraised through the same mechanism and evaluation criteria across sources.

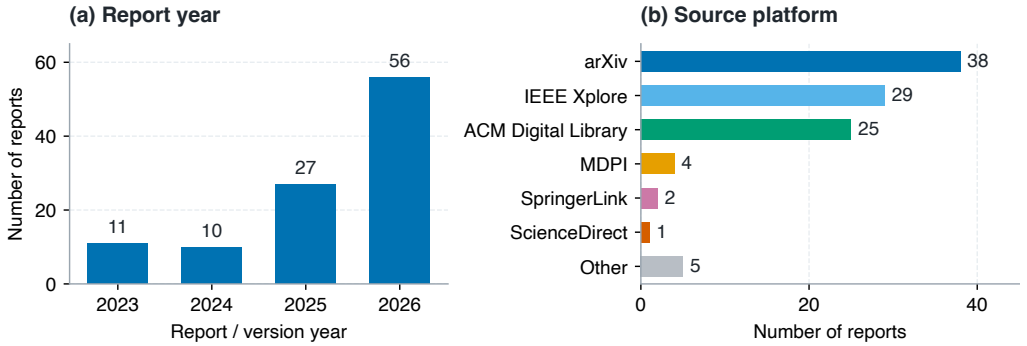


Fig. 2. Year and source platform of the 104 included reports.

3.3 Classification and evidence extraction

Classification records the software task, the graph mechanism, and how the method uses observations to guide subsequent work. The reviewed methods include fixed pipelines, workflows that revise actions after feedback, developer-led analysis, and reusable graph tools or memory services. For each configuration, we record the available observations, who selects the next operation, and whether the response changes retrieval, an artifact, a plan, or retained knowledge. These descriptions support comparisons across the 104 reports.

Graph use and workflow control are recorded separately. A fixed retrieval pipeline can supply evidence to an agent, while a graph service may leave action selection to its client. A workflow can also revise an answer after feedback without updating the graph.

For report-level counts, a model-selected-operation category requires at least one disclosed configuration in which the model selects a subsequent tool, query, delegation, or termination decision using an observation. Predetermined correction routes are recorded separately, even when an LLM generates a revised artifact. The remaining categories distinguish developer-guided use, reusable services, fixed pipelines, and insufficiently specified control. A report receives one counting

category, while configuration notes retain differences within the report; CODENS's fixed and agent modes, for example, are not collapsed into one control claim.

The extraction covers source artifacts, graph schema, construction and identity resolution, graph access, context assembly, feedback, update policy, and software task. Statements are linked to physical pages in the inspected papers. Undisclosed mechanisms remain unclassified; explicit statements that a component is absent are recorded separately. Bibliographic metadata were checked against arXiv, Crossref, or publisher records.

Two reviewers independently read each report and extracted and coded the evidence. They compared their assessments and resolved differences through discussion before recording the final assignments. The codebook, report-level decisions, and supporting evidence provide traceability for the synthesis, which compares relationships and actions within tasks and shared interventions across tasks. Supplementary LLM-assisted checks examined configuration-level feedback and seven evaluation-design criteria for all 104 reports; a further model-assisted pass rechecked 30 stratified reports and 21 targeted cases.

3.4 Decision rules for intervention roles

Roles are assigned by what graph knowledge changes in the workflow. *Evidence access* supplies artifacts, candidates, paths, or relational context for a decision; the access mode distinguishes retrieval from directly supplied graph context. *Constraints and planning* determine allowed outputs, dependencies, or action order. *Training supervision* uses graph knowledge to construct or select downstream training examples, targets, preferences, or graph-conditioned rewards. *Verification* uses graph facts or derived rules as a correctness or consistency criterion. *Persistent state* updates graph-backed knowledge, linked artifacts, or graph-indexed search state for reuse in later iterations or tasks. A stored graph alone establishes neither persistent-state management nor an agent loop.

Role assignment follows the operation receiving graph knowledge. APIKG4Syn samples paths to construct training examples, whereas KGCompass supplies paths to the repair model at task time [55, 115]. These uses receive training-supervision and evidence-access roles, respectively. Constraints shape candidate generation or action order; verification checks candidates against graph-derived criteria. SemanticForge's SMT-guided pruning performs both operations during decoding [128]. Checks used only to build the knowledge graph are recorded under construction quality.

The coding is non-exclusive. VulReaD combines training supervision and inference guidance; RepoDoc combines context access, dependency-based ordering, and documentation maintenance [67, 111]. External test execution is coded separately from graph-based verification, while validation of extracted triples belongs to construction quality. Graph-construction foundations may therefore have no downstream role assigned. Proposed mechanisms and limited-disclosure reports are marked separately, while an unassigned role means that the reviewed evidence does not establish it.

4 RQ1: Graph Knowledge Across the Agent Workflow

4.1 Knowledge sources determine what the graph can explain

Knowledge needs vary with the engineering activity. Requirements analysis can draw on standards, as in SEER; design understanding can draw on recorded decisions and rationale, as in Kantara [21, 88]. During implementation, source code provides declarations and references, package metadata supplies dependencies and versions, and API documents supply signatures and usage constraints [7, 43, 60, 72]. For maintenance, development artifacts additionally record issues and past changes, which KGCompass links to code [115]. The source therefore determines whether a graph can explain an intended constraint, a structural dependency, or the history relevant to a change.

DepsRAG illustrates the value of a deliberately narrow source and schema. It obtains package dependencies through the Deps.Dev API and maps structured responses into package nodes and dependency edges [7]. The graph supports dependency reasoning without requiring an LLM to rediscover relationships already available in machine-readable form. RepoChat instead represents development activity obtained from repository metadata, allowing conversational access to relationships among repository artifacts [2]. The contrast shows why source selection should precede choosing an extraction model.

KGCompass crosses these source boundaries by linking development artifacts to files, classes, and functions [115]. The resulting paths can connect an issue description to code through historical context. Such connections are particularly relevant when the user's description and the implementation use different vocabulary. Their usefulness nevertheless depends on the correctness and temporal availability of the links; additional sources can introduce outdated or unavailable evidence as well as helpful context.

Sources beyond code introduce distinct units of evidence. Kept extracts relations from project documents with MinIE and from Java code with JavaParser, then connects these representations for localization [52]. TAAF builds query-specific temporal subgraphs from recorded execution states, whereas AetherLog organizes logs with root-cause knowledge [19, 26]. The former is bounded by the executed intervals; the latter also depends on which diagnostic labels were available when the graph was built. Security substrates such as VulKG integrate vulnerability, product, vendor, and exploit records, supplying relationships that a repository parser cannot recover [120].

4.2 Graph construction determines which relations are available

Parsing recovers declarations and syntax; identifying an imported or dynamically dispatched callee additionally requires resolution. SGAgent separates AST-based entities from the cross-file relations used in repair [125]. Codebase-Memory uses definition registries, import-aware resolution, type-related processing, and fallback matching, attaching confidence values to the different routes [102]. These values expose the matching route; probabilistic calibration requires a separate assessment. Parsing coverage and relationship-resolution accuracy measure different capabilities.

LLMs often enrich a structurally derived graph. RANGER adds descriptions to parsed code; CKGFuzzer associates semantic summaries with functions and files recovered through program analysis [91, 112]. PtrTrans derives pointer-use relationships and Rust-oriented annotations through static analysis and rules before translation [122]. In contrast, SysKG-UTF uses an LLM to extract bug-report sections and atomic user steps before clustering and linking them [95]. Here, implementation and migration draw on program structure and pointer relations, while testing also draws on API descriptions and reported user actions. Program analysis and language-based extraction supply different parts of that task knowledge.

Textual sources add provenance and review choices. Kantara extracts decisions and rationale and scores their relations, while APIMig combines document heuristics, pattern matching, LLM extraction, and human review for API evolution knowledge [21, 43]. Retaining these origins allows structural facts, extracted statements, and inferred relationships to receive appropriate checks within one graph.

Construction choices affect both relation meaning and coverage. The AST-versus-LLM comparison contrasts typed Java relations with generated dependency links, so the alternatives differ in schema as well as extraction [18]. The local-first method derives call links from generated design descriptions; LogicLens links parsed code to extracted domain entities; GraphShield generates vulnerability and contract relations with an LLM [64, 100, 130]. These links support conceptual navigation, but their provenance differs from statically resolved references. CA-SAF's edge-target

membership check can reject references to missing entities without establishing that the remaining semantic relationships are true [65].

Several studies make construction quality the main contribution. Explore-Construct-Filter develops schema guidance and filters API triples; its related verification report additionally evaluates recommendation using selected triples [96, 97]. KG4CraSolver combines prompted classification and span extraction to recover crash context and solutions, while SoapOperaTG uses linguistic extraction and step clustering before composing test scenarios [24, 94]. SSCKG checks teacher-generated binary semantics against code-property relations before distillation, and the dynamic vulnerability graph filters and merges extracted threat knowledge [58, 69]. These checks improve the knowledge supplied downstream; they should be assessed separately from checks on a generated patch or test.

4.3 Graph roles describe where knowledge influences decisions

The five roles locate different intervention points rather than different kinds of system. Graph paths select repair context in KGCompass but select training examples in APIKG4Syn; domain facts check candidate paths in EntailLLM; dependency links identify documentation to refresh in RepoDoc [55, 66, 111, 115].

Across the 104 reports, the assigned counts are evidence access 92, constraints/planning 30, training supervision 6, verification 13, and persistent state 22. The full-collection ordering survives the recorded proposal, domain-boundary, and related-report sensitivity checks.

Graph-supported work can receive feedback from checks outside the graph. MPC-Coder uses compiler and model-checker outcomes; TestAgent uses test execution and coverage [126, 129]. These observations assess generated artifacts, whereas the graph-verification role requires graph facts or derived rules to supply the checking criterion. Similarly, retaining graph-backed state does not by itself establish that all dependent knowledge is refreshed after edits.

The comparison needed to evaluate a role follows the operation it changes. For RepoChat, holding source records and the answer model fixed while varying the query interface would test evidence access. For KNSE-Code, rule selection should be separated from rule representation; for APIMig, the same API facts can support comparisons of evolution links and change propagation [2, 31, 43].

Training and checking require different controls. APIKG4Syn's path selection can be varied under a fixed training budget, whereas VulReaD's training-data grounding should be separated from its inference retrieval [55, 67]. EntailLLM motivates separate comparisons of label injection, path realization, and logical checking [66]. RepoDoc's maintenance role can be tested by comparing selective and full regeneration for the same source changes [111].

Shared roles can conceal different maintenance obligations. PtrTrans orders translation units, uses compiler diagnostics for bounded repair, and retains a Rust graph; RepoDoc propagates commit changes to documentation and checks cross-references [111, 122]. Both select and order work through relationships. Translation must additionally establish covered behavioral preservation; documentation must establish affected-content coverage and consistency. The trigger, changed artifact, and oracle determine what the shared role can guarantee.

4.4 Graph representation determines which questions can be answered

The same real-world object can appear as a node, an attribute, or a text span. An API parameter represented only as a string cannot support the same queries as an entity connected to a type, version, and constraint. Increasing granularity, however, also increases storage, linking work, and retrieval branching. Representation should therefore be compared against the queries it enables rather than against its node count.

Identity is especially important across repository boundaries and versions. Names alone are insufficient when classes share a short name, APIs change signatures, or a package exists in several

ecosystems. A practical identity model may use fully qualified names, source locations, repository revisions, and library versions. Versioned APIs in APIMig and import-aware symbol resolution in Codebase-Memory provide concrete examples of why these distinctions matter [43, 102]. These examples motivate explicit identity rules for comparisons across repositories and versions.

APIMig explicitly represents evolution between API versions and composes changes across a migration path [43]. This is different from a graph that merely stores the latest documentation. AgenticSZZ represents commits and changed code entities, but its reported temporal ordering is based on commit dates rather than the full Git parent-child DAG [92]. That distinction limits what a temporal edge alone establishes: chronological precedence is useful evidence for candidate search, but it is not a complete representation of branch ancestry or a proof of causal responsibility.

Domain ontologies also determine what can be checked. JDQuery maps Java facts to an ontology and expresses defect patterns as graph queries; the specification-generation method represents functional and data relationships for completeness and consistency checks [34, 108]. IntelliTest links procedures to states, resources, and constraints, enabling sequence validation before execution [99]. In these systems, the represented rules define the reach of verification: an unrepresented condition cannot become a checking criterion merely by adding more instances.

4.5 Retrieval defines how agents access evidence

The retrieval studies illustrate four mechanisms that can be combined within one system.

Query-language access. CodexGraph enables an agent to construct and execute graph queries [60]. DepsRAG similarly exposes graph-query functionality and retrieves schema information before query generation [7]. This interface supports compositional constraints, but it creates multiple failure modes: a query can be invalid, executable but semantically wrong, overly broad, or correct but too expensive to interpret. Query execution success should therefore be measured separately from answer correctness.

Anchor, expand, and refine. RepoHyper retrieves initial candidates, expands them through a repository semantic graph, and refines the context [83]. Its analysis of expansion strategies motivates treating neighborhood size and selection policy as substantive parameters. Neighborhood size and refinement determine which structural evidence reaches the generator.

Artifact-to-code paths. KGCompass uses relationships among issues, pull requests, and program entities to guide localization and supplies entity paths during repair [115]. Here the path is both a retrieval object and a representation of why a candidate may be relevant. Evaluation should separate finding the target from the additional effect of exposing the path to the generator.

Adaptive exploration. RANGER first attempts graph queries and uses Monte Carlo tree search when direct access does not return a result [91]. The distinction between explicit code-entity queries and natural-language queries motivates adaptive routing. It also creates a cost question: fallback exploration must be assessed with its additional actions and model calls, not compared as if it were a single retrieval operation.

RepoGraph provides another point of comparison by offering repository-level code structure as an augmentation for software engineering methods [71]. This places repository structure at the interface between evidence access and downstream engineering tools.

Retrieval and context assembly jointly determine what the model can use. RepoHyper separates expansion from refinement; SWE-agent studies the presentation of tool output and history [83, 117]. Code snippets, signatures, relation labels, path explanations, and tests consume different amounts of context and support different actions. Compare anchor selection, constrained expansion, pruning, and serialization under a fixed token budget. Holding retrieved code and budget fixed while varying path information isolates relational presentation; holding the agent fixed while varying selection isolates evidence access.

4.6 Graph knowledge provides supervision for model training

Graph-derived supervision changes the model before a user task begins. APIKG4Syn samples API combinations to synthesize fine-tuning examples, whereas cooperative mashup recommendation uses Node2Vec embeddings as regression targets for its text encoder [55, 124]. DomAgent trains retrieval-tool use with graph-conditioned examples and relevance rewards; CKG-LLM uses supervised learning and preference optimization for graph-query generation [49, 106]. SemanticForge reports synthetic dependency queries and graph-conditioned generation rewards [128]. The supervision may therefore teach API composition, representation alignment, or an access policy, rather than simply add facts to a prompt.

Training a graph encoder is a different use. VulnScopper supplies LLM text embeddings to an inductive graph predictor, and BugRadar learns report–file associations for localization [6, 110]. Neither operation by itself shows that graph knowledge supervises a downstream language model. Kept instead injects document/code triples into the encoder input through visibility-masked attention [52]. These distinctions matter when attributing a result to better representations, additional training signals, or runtime evidence.

4.7 Workflow control determines who acts on graph evidence

The control ledger distinguishes 50 reports with fixed pipelines, 16 with prescribed feedback routes, and 14 that establish model selection of a subsequent operation. Seven describe developer-guided use, three provide reusable graph services, and 13 do not establish the operation-selection mechanism: 11 lack sufficient control evidence and two describe feedback at the architectural level. One further report is the physical-domain boundary comparison. The 16 prescribed-feedback reports comprise 13 artifact-adaptation workflows, two bounded corrections, and one retrieval fallback. These mutually exclusive counts sum to 104.

The distinction is visible within individual studies. CODENS exposes fixed standard/multi-hop modes and an agent mode that chooses another graph tool or an answer after observing results [41]. The cloud-diagnosis study separately evaluates Straight-Shot, ReAct, and Plan-and-Execute configurations [87]. Graph access can therefore remain available while the decision about when and how to use it changes. A tool service such as Codebase-Memory leaves that decision to its client; providing an interface alone does not establish a model-controlled workflow [102].

Graphs can influence the order and scope of actions. CodePlan combines incremental dependency analysis, change-impact reasoning, and adaptive planning to coordinate repository-level edits [11]. Its dependency analysis provides a structural basis for planning. APIMig likewise uses evolution knowledge and project-level change chains to identify code affected by an API migration [43]. In both cases, relationships help determine which work must follow a change.

Feedback changes the next action. APIMig orders affected callsite blocks before prompting for migration, while KCoEvo constructs evolution paths that guide generation [39, 43]. PtrTrans additionally connects a failed compilation to a new repair: it identifies the affected unit, retrieves linked Rust units and pointer annotations, asks the LLM for a patch, and recompiles [122]. The difference is the transition from an observed outcome to a revised action. The companion control ledger distinguishes these transitions by observation and controller.

Control depends on the workflow stage. SGAgent's localizer and suggester expand context through tool interaction; its base repair stage uses test outcomes to rank patch candidates. The SGAgent-Vul extension additionally feeds failed tests into repair and can escalate to re-suggestion and re-localization [125]. LingmaAgent similarly distinguishes dynamic repository retrieval from its execution-feedback variant [61]. KGCompass describes test-based patch ranking, whereas DSrepair uses a failed initial execution to construct a feedback-conditioned correction prompt [72, 115].

These mechanisms differ in both what receives feedback and who selects the next action. A bounded correction, repeated artifact repair, and adaptive tool selection should be described separately.

The observation determines the reach of the correction. CuFuzz returns compiler diagnostics and API-identifier checks to the harness generator; MPC-Coder returns syntax errors and bounded model-checking counterexamples; ReProAgent combines execution with review of whether the failure matches the issue [27, 127, 129]. Each exposes different errors. Compiler acceptance addresses language constraints, a counterexample addresses the specified property, and a reproducing test addresses an observed behavior. The graph supplies context or constraints to the correction, while the external tool supplies the observation.

What is retained determines what an update can guarantee. Prometheus stores retrieved context in a separate multi-vector memory and consults it before expanding the graph; a transaction keeps memory records, context units, and their links consistent [75]. Codebase-Memory instead responds to changed-file hashes by replacing file-level graph records and recomputing affected communities [102]. The first mechanism preserves reusable evidence; the second refreshes a structural index. Connecting them requires invalidating cached evidence when the source records on which it depends change.

Derived artifacts introduce another propagation problem. RepoDoc classifies commit changes, propagates their impact through code and documentation relations, regenerates affected material, and checks cross-references and consistency [111]. PtrTrans maintains linked Rust context during translation, while WebDesignIter refreshes implementation and design information after validation [79, 122]. Graphiti MCP accumulates development episodes and temporal relations across sessions [116]. These mechanisms maintain different objects: source structure, derived documentation, implementation history, or observations. A useful comparison must identify the update trigger, affected dependencies, and acceptance check for each object.

Feedback-driven artifact revision should not be interpreted as autonomous correction of the knowledge source. TestAgent can select updates to graph-linked tests, whereas a failed ReProAgent test triggers investigation and test revision; neither observation alone establishes repair of an incorrect graph relation [126, 127]. For model-selected operations, evaluation must additionally record what evidence was available before each choice, which alternatives were permitted, and when the controller stopped. A quality gain can otherwise combine a better access policy with more observations or a larger computation budget.

RQ1 connects how a relationship is established, which operation consumes it, and what invalidates it. These choices explain different forms of engineering support: rationale makes design decisions inspectable, code dependencies guide implementation and repair, and change links identify documentation to maintain. Each use imposes its own checks and update obligations.

5 RQ2: Software Engineering Applications and Task Conditions

The task synthesis covers 103 software or adjacent operational reports across requirements and design, implementation, testing, deployment preparation, operation, and maintenance. Its groups comprise repository understanding (16 reports), requirements and design (8), API discovery and training (8), generation and migration (10), configuration and control (7), localization and repair (11), security and auditing (21), testing and reproduction (10), and documentation and operations (13). These are 104 group assignments over 103 reports: HybridCode contributes to both repository understanding and documentation [119]. The physical-welding boundary report receives no software-task assignment [30].

Evidence access occurs in every report assigned to repository understanding, generation and migration, configuration and control, localization and repair, and documentation and operations. Other roles follow the demands of the task: constraints and planning appear in seven testing

reports, six generation/migration reports, and five configuration/control reports. Security and auditing account for seven graph-verification assignments, while documentation and operations include seven persistent-state assignments. Three API-discovery/training reports use graph-derived supervision. These non-exclusive assignments connect the information supplied by a graph to the decisions and artifacts discussed below.

5.1 Repository context supports program understanding

Repository questions require history, resolved code structure, or conceptual explanations. RepoChat and its extended report query repository metadata under a schema; RepoWise routes question types across graph/vector access, and the design-system assistant exposes parsed relationships through MCP [1, 2, 12, 42]. These methods define the evidence operations on which interactive exploration depends.

LogicLens chooses project, entity, graph, and source tools; Prometheus revises subqueries using accumulated context [75, 100]. Codebase-Memory's symbol resolution and RANGER's fallback search supply useful comparisons for what those operations return [91, 102]. The AST-versus-LLM and code-RAG studies compare alternative construction/access choices on shared repositories [18, 73]. Together they separate evidence representation from the controller choosing further exploration.

Explanatory tasks add another requirement: code relations must connect to the user's concepts. The local-first method, CA-SAF, and HybridCode combine structural context with descriptions; Javapers, XCoS, and the cryptography tutor support pattern queries, concept explanations, and instructional comments [29, 64, 65, 89, 103, 119]. Thus, symbol navigation and conceptual explanation are complementary access needs, with different source and alignment checks.

Our complementary repository-question experiment (Section 6.5) illustrates this distinction in program comprehension. A Bash-based agent scores above one-pass source-context answering, whereas adding graph tools to a source-browsing agent produces no consistent score improvement across the sampled questions. Graph access and adaptive source exploration are therefore separate comparison factors.

The interfaces differ in who initiates further investigation. RepoWise routes questions to structural, semantic, or fused evidence before answer generation; RepoChat translates a question into a query and verbalizes its result. The extended RepoChat report restricts query vocabulary through the supplied schema but leaves error-conditioned query revision to future work [1, 2, 42]. The design-system assistant demonstrates graph-tool access in a bounded internal codebase, while Javapers supports human inspection of typed neighborhoods and pattern-query results [12, 89]. These applications make repository evidence accessible through different combinations of routing, querying, and user control.

5.2 Design knowledge makes engineering intent explicit

At this stage, graph relations connect an intended product to standards, design rationale, and responsibilities. Specification generation checks functional/data relationships; requirements retrieval shortlists potentially conflicting pairs through shared semantic roles [78, 108]. SEER adds model-directed assessment of context sufficiency and expert review of derived sustainability requirements [88]. The connection is between identifying applicable evidence and deciding whether further evidence or human interpretation is needed.

Design-pattern recommendation supplies scaffolding but underdiscloses graph operations [9]. WebDesignIter links design memory to evolving code, while the two Kantara reports recover decisions and rationale from commit history [21, 22, 79]. These mechanisms make design intent available for later implementation and review. Developer-assignment graphs similarly connect

issues to workload and history [46]; their historical commit features must be restricted to information available at the assignment time. Requirements and design therefore need traceable intent and a valid temporal boundary, not simply more code context.

The two Kantara reports contribute at different stages: the initial study classifies commit sentences and preserves their order and provenance for rationale queries; the later study extracts decisions and rationale and scores relationships for inconsistency analysis [21, 22]. This knowledge can explain why a design changed, provided that the extracted statement remains traceable to its originating commit.

5.3 API knowledge helps developers select suitable implementations

API support operates at selection, clarification, and learning. KGE4AR ranks analogical APIs, cooperative mashup recommendation injects graph embeddings into a text encoder, and MLTaskKG filters by task and environment [56, 57, 124]. KPL uses path examples to clarify intent under user-controlled continuation [35]. Explore-Construct-Filter and its related verification report establish schema/triple acquisition and filtering, with downstream recommendation in the latter [96, 97].

DomAgent uses SearchKG and SearchCase to acquire knowledge and request follow-up evidence [106]. APIKG4Syn instead uses uncertainty-guided selection to construct training examples [55]. The comparison separates knowledge acquisition before deployment from evidence acquisition during a task. An agent's ability to request another API fact depends on both the available knowledge and its learned access policy; Section 6.3 distinguishes these sources of benefit.

The query also defines the recommendation problem. KGE4AR starts from an API whose analogue is sought; MLTaskKG connects a task and environment to candidate libraries; KPL uses each answer to choose a subsequent clarification aspect [35, 56, 57]. A useful recommendation must consequently preserve the relevant behavior, satisfy environmental requirements, or resolve the user's uncertainty, depending on the starting information.

5.4 Dependencies guide changes across a repository

Generation and migration expose constraints that remain necessary regardless of who controls iteration. Graph expansion/reranking, pruned programming records, and version-resolved cross-repository retrieval supply implementation context [10, 48, 90]. SemanticForge constrains decoding; dependency-ready generation orders components and revises structurally invalid outputs [5, 128]. These methods locate constraints before or after candidate generation, providing concrete alternatives to unrestricted generation followed by repair.

Version changes require identities and ordering beyond a current call graph. KCoEvo aligns API entities across versions, APIMig links API evolution to project changes, and CodePlan propagates dependency-aware edits [11, 39, 43]. Porting adds language-specific acceptance: ArkAdapter uses compilation and differential tests, PtrTrans combines pointer/dependency ordering with a retained Rust graph, and Devito translation uses static checks and model judgments [33, 122, 131]. These methods illustrate the planning and checking obligations that an editing agent must coordinate.

CrossCoder includes installed-library code alongside the current repository, making dependency versions part of the retrieval context [48]. By contrast, KCoEvo explicitly aligns entities across API versions to construct evolution trajectories [39]. The two approaches address different gaps: finding the implementation available in the current environment and identifying the changes needed to reach another version.

5.5 Domain rules constrain generated artifacts

Configuration requires both a suitable design and valid instantiation. KNSE-Code selects relevant aviation rules; ConMover supplies Move concepts and candidate plans, then uses visible test failures

for bounded debugging [31, 40]. Terraform knowledge injection models provider arguments and cross-resource relations, whereas the dual-layer method also models scenario/subtask/resource intent [68, 121]. Code-to-execution GraphRAG applies retrieved context and error feedback to deployment artifacts [47]. These mechanisms support deployment preparation through domain and dependency constraints.

For industrial control, assisted PLC generation retrieves complete trigger/branch/output templates, while MPC-Coder can reformulate insufficient queries and revise code after verification reports [70, 129]. The comparison identifies two independent design choices: preserving a complete control-logic unit during retrieval and choosing an action after its instantiated code is checked. Schema compliance, deployment intent, and plant behavior remain the relevant acceptance targets; Section 6.4 compares their measured outcomes.

Rule delivery also varies within a method. KNSE-Code first injects its entire set of 225 validated triples, then introduces a top-k filter based on keyword overlap and semantic similarity [31]. Assisted PLC generation retrieves complete trigger, branch, and output templates, after which engineers configure instance parameters [70]. These examples distinguish selecting applicable obligations from correctly instantiating them in an executable artifact.

5.6 Related artifacts guide defect investigation

Defect handling progresses from candidates to edits and, for historical analysis, causal attribution. BugRadar ranks report–file links, JDQuery matches declarative defect patterns, and Kept embeds document/code triples [34, 52, 110]. KG4CraSolver ranks historical solutions, BKG repair retrieves bug/fix pairs, and DSrepair combines API facts with execution-localized failures [14, 24, 72]. LIDL adds dependency-based hypothetical scoring for integration defects [98]. These methods provide alternative evidence and ranking procedures for a repair workflow.

LingmaAgent and SGAgent use observed context to continue repository exploration; KGCompass tests path presentation after candidate localization [61, 115, 125]. This comparison distinguishes missing candidates from missing relationships among candidates. AgenticSZZ instead selects graph queries, diff reads, and stopping decisions to identify a bug-inducing commit [92]. Its date-order edges must be distinguished from Git parent ancestry when judging temporal explanations. Across repair and attribution, the graph's task is to narrow the relevant change and explain its dependencies; the acceptance target is defined by the requested output.

Our complementary localization experiment shows why this distinction matters. Graph queries can return a reference file while the agent ranks another location first or expresses the function identity incorrectly. Graphify nevertheless improves first-ranked function hits on five cases and loses on two relative to source browsing. These observations concern candidate selection for maintenance, not successful patch generation (Section 6.5).

Historical context requires a task-time boundary. BKG retrieves previous bug/fix pairs, while BugRadar combines graph-based report–file scores with lexical and historical signals [14, 110]. The latter uses random report partitions for graph learning and a later chronological split for its final aggregator; the final split alone does not establish historical isolation throughout the pipeline. LIDL's validator instead scores hypothetical fixes and compares candidates without executing a repair [98]. Its output is evidence for localization, whose correctness must be judged separately from patch behavior.

5.7 Security analysis requires traceable evidence

Security uses graph knowledge for candidate discovery, evidence checking, and retention. VulnScopper predicts affected-product links; VulKG-RAG and the repair-recommendation method retrieve vulnerability/code associations [6, 8, 37]. Compact vulnerability graphs and dynamic multi-source

construction supply risk relations and retained updates [58, 120]. CKG-LLM checks query slots against its ontology and learns query preferences; VulReaD uses graph-guided training preferences and inference context [49, 67]. Their interventions occur at different points in the detection pipeline. SCPatcher instead returns failed Slither checks to a second patch-generation stage [50].

The checking mechanisms are particularly relevant to agent integration. KAVE's two reports separate path discovery from propagation/sanitization checks; SSCKG checks generated data-flow claims before semantic-lifting training [69, 84, 85]. GraphShield combines semantic and contract-structure retrieval, CLEAR checks hypotheses against causal preconditions, and EntailLLM realizes connected call paths before logical entailment checking [66, 123, 130]. These methods demonstrate checking operations that could inform an agent's next action.

EvoPoC combines exploit generation with verification-conditioned retries, and ASTRA retains graph-indexed exploration state for assistant safety testing [53, 113]. Policy auditing instead aligns privacy statements with static flows, while PowerReviewer and PowerRecall connect domain specifications to code review through constraints, retrieval, and expert corrections [15, 51, 62]. Durinn proposes finding-centered retrieval; CyKG-RAG connects public security knowledge with local logs [44, 63]. The resulting evidence spans specific checks and operational workflows, rather than one interchangeable security-success measure.

Security graphs represent different objects even when they share vocabulary. VulnScopper predicts missing affected-product links; VulKG exposes exploit neighborhoods; SSCKG recovers binary semantics [6, 69, 120]. CLEAR structures a vulnerability hypothesis around entry points, preconditions, and root causes, then checks it against retrieved context [123]. ASTRA instead represents an assistant's safety-test space and updates sampling priorities across rounds [113]. Graph connectivity can guide all these searches, but product exposure, feasible vulnerable behavior, and a safety-test finding require different evidence.

5.8 Test construction depends on relevant behavior

Testing uses relations to select scenarios, constrain their composition, and define expected behavior. SoapOperaTG and SysKG-UTF recombine bug-report steps; IntelliTest adds procedure constraints and expert-reviewed retention [94, 95, 99]. TestAgentX selects regression tests through code/test/bug relations, while KGIT derives expected answer relations from graph inference [101, 104]. These methods supply selection and oracle mechanisms that an agent controller can invoke.

TestAgent retrieves context and chooses whether to accept, discard, update, or augment graph-linked tests; ReProAgent revises inputs and assertions using execution and issue correspondence [126, 127]. KITE provides a complementary acceptance gate: an enhanced suite must pass correct code, detect the injected bug, and preserve earlier detections [107]. The engineering objective therefore determines the useful feedback: coverage for exploration, fault discrimination for regression, and issue-specific fail-to-pass behavior for reproduction.

CKGFuzzer retains successful drivers and targets branch coverage; CuFuzz records API-edge coverage to guide later sequences [27, 112]. These states avoid repeated exploration within a run. They offer a different maintenance problem from refreshing API facts after a library upgrade, discussed in Section 6.7.

KGIT illustrates a graph-derived test oracle: inference over background facts supplies related questions and expected answers for QA software [104]. IntelliTest derives valid procedure sequences from domain constraints and admits new procedure knowledge after expert review [99]. Their reusable objects are expected relations and reviewed procedures, respectively; neither should be equated with merely retaining a generated test file.

5.9 Maintenance requires knowledge that follows software changes

Documentation exposes the difference between revising an answer and updating its sources. LEDGE retrieves dependencies, CLAIR uses critic feedback, and HybridCode supports user-guided regeneration [25, 76, 119]. CODENS replays pull requests into semantic attributes and selects graph tools in agent mode; RepoDoc propagates source changes through code–concept–document dependencies [41, 111]. Together they connect interactive explanation to the separate obligation of maintaining the material explained.

Runtime analysis requires observations of the relevant execution. AetherLog retrieves accumulated diagnostic knowledge; TAAF constructs temporal graphs for query-relevant trace intervals [19, 26]. The cloud-RCA study compares full-context diagnosis with ReAct and plan-revising graph access, exposing how inspection affects hypotheses [87]. The performance-bug framework proposes connecting diagnosis, interventions, measured outcomes, and retained knowledge, with implementation details still limited [86].

For continuing maintenance, DepsRAG decomposes dependency questions and retries failed queries. CCCE supplies event-dependent impact closures and review gates; Graphiti MCP retains temporal episodes; Code Digital Twin proposes version-scoped knowledge and writeback [7, 77, 82, 116]. These relationships connect an agent's immediate investigation to longer-lived evidence. The required refresh follows the source dependencies of each retained object, not simply the completion of the current task.

The available source model determines the maintenance work. CLAIR begins with an existing low-code metamodel, whereas LEDGE parses dependencies and retrieves supporting documentation [25, 76]. The former depends on metamodel synchronization; the latter must preserve the relation between source code and generated descriptions. CCCE describes event-triggered dependency closures and risk gates, while Code Digital Twin proposes version-scoped context and accepted-change writeback [77, 82]. These designs specify useful maintenance responsibilities, although their longitudinal empirical support differs from the implemented update evaluations discussed in RQ3.

6 RQ3: Evaluation Practices and Evidence

We examine how studies evaluate graph support, what their comparisons establish, and how they account for cost. The literature results and complementary repository experiments retain their respective task settings and comparison conditions.

6.1 Evaluations provide uneven control over graph effects

Table 1 summarizes seven evaluation-design criteria across the 104 reports. Counts describe reported designs rather than a combined quality score, and related reports do not represent independent experiments. A within-study contrast supports conclusions about the intervention actually varied: a fixed-candidate path comparison can establish a narrower effect than a whole-pipeline comparison. Mechanism descriptions establish what a system does, while comparative results assess the tested configuration.

A common source pool can still permit differences in enrichment or access. Token matching requires the total task-level input and output budget; a context cap supplies only a partial control. Existing software artifacts include requirements and runtime records as well as code, without implying production deployment. The update criterion requires both quality and cost after a source change, while intermediate assessment concerns the evidence or trace leading to an outcome.

Table 1. Evaluation-design criteria across the 104 reports. Partial evidence is reported separately; an unconfirmed criterion does not establish that it was absent.

Criterion	Confirmed	Partial
Common raw-source comparator	72	14
Equal total-token budget	0	4
Component contrast	66	4
Explicit outcome oracle	96	6
Existing software artifact/task	93	5
Source-change quality and cost	2	5
Intermediate evidence/trace assessment	40	5

Component evidence differs from *structural attribution*. Common-source coding requires one comparison with a shared source pool, including critic, prompt, or backbone comparisons. DepsRAG's critic ablation retains its graph, whereas AgenticSZZ's graph-free variant loses candidate reach and MPC-Coder's KG removal withholds domain constraints [7, 92, 129]. The 66 reports with component contrasts cannot be counted as demonstrations of topology-specific benefit. Across the collection, the available contrasts therefore leave relational structure, context selection, and pipeline effects incompletely separated. The absence of a confirmed strict total-token match further limits attribution in LLM comparisons. Across the 104 reports, 96 establish an outcome oracle and 40 separately assess intermediate evidence or traces. MPC-Coder's manual snippet-relevance judgments illustrate this additional assessment. Outcome scores alone do not establish whether correct results relied on correct intermediate evidence.

Two reports establish the combined update criterion: RepoDoc measures recall and cost for documentation updates around real commits; SemanticForge reports commit-based indexing costs and a stale-graph ablation [111, 128]. CrossCoder measures update latency and separately tests version compatibility; IntelliTest reports longitudinal procedure maintenance [48, 99]. These provide partial evidence. Prometheus evaluates memory reuse and CODENS replays PR history, but neither establishes invalidation of all dependent cached evidence [41, 75]. The unresolved requirement is coordinated validation of graphs, caches, plans, and derived artifacts after edits.

6.2 Evaluation measures depend on the software task

Repository answers require evidence checks. RepoChat's tool report checks 40 questions against one repository; its extended report adds prompt comparisons over fixed repository graphs and questions [1, 2]. RepoWise evaluates routing and answer quality, although its measured retrieval subset and estimated full-benchmark component results must be distinguished [42]. The internal design-system assistant uses reference answers and an LLM judge; LogicLens uses expert ratings of accuracy, completeness, and coherence [12, 100]. These assessments concern the returned explanation, so they complement rather than replace direct retrieval checks. The local-first study uses expert-identified files/functions, the AST-versus-LLM study compares construction/access alternatives on pinned repositories, and the code-RAG comparison scores answers to 100 Flask questions [18, 64, 73]. Javapers compares recovered pattern classes with a reference, while the cryptography tutor separates retrieval metrics from instructor-rated answers [29, 89].

Requirements need explicit quality criteria. The specification-generation study checks correctness, completeness, and consistency rules over requirement graphs; the requirements-retrieval study separately measures candidate selection and conflict classification [78, 108]. The initial Kantara report evaluates sentence classifiers on annotated commits, and the later report

assesses decision/rationale extraction and relations [21, 22]. SEER evaluates derived sustainability requirements and catalog-grounded reasoning with expert input; developer assignment evaluates combinations of issue, commit, and developer features [46, 88]. WebDesignIter extends the evaluation to sequential implementation tasks, using tests and regression errors to assess evolving designs [79]. These outcomes span extraction, recommendation, and implemented behavior; an improvement at one level does not establish improvement at the others.

API recommendations must satisfy the query. KGE4AR evaluates ranking against validated analogical API pairs and varies reranking, embeddings, and knowledge types. Cooperative mashup recommendation uses held-out invocation labels and removes graph-to-text knowledge injection [56, 124]. MLTaskKG separately checks sampled triples and user satisfaction with recommended libraries, while KPL evaluates clarification units and a pathfinding variant that uses previous-round answers [35, 57]. Explore-Construct-Filter measures extraction and filtering; the related verification report adds recommendation questions while retaining separate construction-quality checks [96, 97]. These designs identify whether errors arise in the knowledge, the interpretation of intent, or the final ranking. APIKG4Syn and DomAgent additionally evaluate generated code after graph-supported learning or retrieval, as examined in Section 6.3 [55, 106].

Generation must fit its environment. Repository-level generation uses EvoCodeBench pass@1, while the inspected Programming KG report evaluates pruning and reranking on HumanEval and MBPP [10, 90]. RANGER combines labeled retrieval and CrossCodeEval completion; CrossCoder tests planning/expansion with repository and installed-library context [48, 91]. KCoEvo evaluates generation against historical API changes, and APIMig compares migration configurations and backbones using API-evolution knowledge [39, 43]. These versioned tasks assess adaptation to a specified target; graph refresh after later changes is a separate experiment. PtrTrans measures compilation and fuzz-derived equivalence, ArkAdapter combines mismatch/API-mapping assessment with compilation and differential tests, and the Devito study compares retrieval strategies alongside translation checks [33, 122, 131]. The strength of a migration result thus depends on the behavior covered by its acceptance checks.

Supporting repository methods make these distinctions explicit. RepoHyper scores context retrieval on RepoBench-R and completion on RepoBench-P, using exact match and CodeBLEU for the latter [83]. CodexGraph evaluates completion, issue resolution, and repository generation with their respective matching, test, and dependency-recall measures [60]. RepoGraph separately analyzes localization and resolution when added to existing software-engineering systems [71]. CodePlan tests coordinated edits through repository validity checks, comparing planning with baselines supplied with the same types of contextual information [11]. Together these designs show why a retrieved dependency, a matching completion, and a valid multi-file change should be reported as distinct outcomes.

Executable artifacts need domain-specific checks. KNSE-Code assesses functionality, safety, and coding-rule violations on aviation requirements; assisted PLC generation checks required template and function items and analyzes retrieval mistakes [31, 70]. ConMover compares initial generation and refinement, but its reported memory-admission gate also consults hidden tests, making test isolation consequential when interpreting reuse [40]. The dual-layer Terraform method evaluates resource and intent knowledge, complementing the separate technical and intent checks in Terraform knowledge injection [68, 121]. Code-to-execution GraphRAG evaluates Docker execution, dependency imports, and workflow execution across repositories [47]. These checks establish different parts of deployability; MPC-Coder's syntax and bounded model-checking reports address further obligations for control programs [129].

Localization must identify the relevant defect. BugRadar and Kept use historical bug–file labels, varying ranking signals and knowledge-enhanced representations, respectively [52,

110]. JDQuery checks manually inspected and injected defects against declarative patterns and a common-rule PMD comparison [34]. KG4CraSolver evaluates crash-information extraction against annotations, whereas BKG evaluates search relevance separately from patch/reference agreement [14, 24]. LIDL uses bug-patch locations to assess ranked candidates; SGAgent separates localization, patch tests, and tool-use analysis [98, 125]. Such location labels help diagnose evidence selection, while the repair comparisons in Section 6.3 use execution outcomes to establish whether a proposed change succeeds.

Security outcomes depend on the prediction target. VulnScopper evaluates inductive link prediction with a temporal vulnerability split; VulKG tests feature combinations against co-exploitation labels; dynamic vulnerability-graph construction evaluates entity and relation extraction [6, 58, 120]. Each measures a knowledge-building or prediction task. Downstream detection adds different controls: VulKG-RAG varies retrieval reweighting, CKG-LLM compares query-training regimes on held-out contracts, SSCKG ablates binary semantic lifting and graph components, and GraphShield varies staged reasoning while retaining its retrieval channels [37, 49, 69, 130]. CLEAR assesses vulnerable/fixed code pairs and weakness labels while comparing retrieval and verification configurations [123]. These designs distinguish extraction quality, representation learning, and detection decisions without treating their scores as one common security measure.

Security review and repair require additional acceptance criteria. The KAVE research report compares detection on PHP applications with labeled and manually adjudicated cases; the related tool report adds corner-case tests and summarizes application results [84, 85]. PowerReviewer evaluates defect labels and safety false positives against retrieval/component variants, whereas PowerRecall evaluates the relevance of retrieved defect patterns [15, 51]. The repair-recommendation study compares graph retrieval strategies using reference fixes, static checks, and model rubrics; SCPatcher isolates its refinement stage with graph retrieval retained [8, 50]. EvoPoC reports verification-conditioned synthesis and a validation-on/off comparison, while ASTRA compares exploration strategies over a shared simulated test space [53, 113]. The latter comparisons measure testing procedures, not classification accuracy or unrestricted patch correctness.

Tests must distinguish correct and faulty behavior. SoapOperaTG distinguishes developer-confirmed bugs from duplicate, invalid, or unconfirmed findings; SysKG-UTF additionally evaluates extraction and scenario feasibility before human exploratory use [94, 95]. KGIT checks graph-inferred expected answers and manually assesses generated test quality [104]. TestAgent combines compilation, execution, coverage, mutation, and manual bug review, while KITE tests the effect of synthetic-bug feedback with knowledge retrieval retained [107, 126]. CKGFuzzer varies coverage-guided mutation under a shared time limit, and CuFuzz compares mutation using the same generated harnesses [27, 112]. Coverage describes exploration; confirmed bugs and issue-specific fail-to-pass tests provide separate evidence of fault detection, as in ReProAgent [127].

Retained knowledge requires task-specific evaluation. LEDGE compares generated documentation with official material using text similarity; HybridCode compares extracted output facts with reference README/article content [76, 119]. CLAIR adds developer ratings of documentation for low-code microflows [25]. Prometheus assesses context against developer-built references and removes memory, testing, and patch-selection components separately [75]. For runtime questions, TAAF compares graph/schema configurations over recorded traces; AetherLog compares diagnosis backends on shared log datasets and knowledge [19, 26]. Graphiti MCP reports completion and context-relevance judgments, with additional historical information in its graph condition [116]. These studies measure access and use of retained knowledge; source-change validity requires the additional update evidence discussed in Section 6.1.

Some reports supply a mechanism or preliminary setting without a sufficiently specified comparative evaluation. Design-pattern recommendation and CA-SAF leave important corpus or

correctness-adjudication details unclear; the performance-bug framework and TestAgentX underdisclose parts of their scoring protocols [9, 65, 86, 101]. Constraint-guided repository generation describes compilation and structural checks but gives limited per-ablation detail; Code Digital Twin presents preliminary task results alongside a broader maintenance vision [5, 82]. CCCE's enterprise scenarios and CyKG-RAG's query demonstrations illustrate intended use, while Durinn's graph route remains an in-progress extension [44, 63, 77]. These reports inform the design space without supporting the same comparative conclusions as measured interventions.

6.3 Comparisons identify the contribution of graph interventions

Reported comparisons locate benefits at different stages of a workflow. DSrepair evaluates 562 DS-1000 failures with four repair LLMs. Replacing text windows with graph retrieval raises fix rates from 14.77–25.44% to 18.51–29.18%, with paired gains of 2.13–3.74 percentage points (source Table 5) [72]. The windows approximate average graph-result length per keyword, while actual total token use is reported separately. The study also finds that compact API expressions outperform richer parameter/return descriptions across its tested models.

KGCompass instead varies relational presentation after localization. On 300 SWE-bench Lite issues, adding entity paths increases resolved cases from 102 to 108 with Claude-3.5 and from 131 to 142 with Claude-4 (source Table 6) [115]. The candidate list and temperature of 0.0 are fixed, separating path context from candidate localization and multi-patch ranking. XCoS exposes a prerequisite for such context: its linking algorithm achieves 95.2% accuracy on 384 descriptions and 72.3% on 50 code snippets (source Table 2), using different samples with different code-name ambiguity [103]. Background knowledge must first be connected to the relevant code.

Training interventions change what is learned before inference. In APIKG4Syn's HarmonyOS generation setting, random, frequency-based, and MCTS selection achieve 20.37%, 22.22%, and 25.00% pass@1, respectively (source Table 5) [55]. This comparison varies selection within graph-based synthesis. The same table's 9.26% text-RAG result omits fine-tuning, changing the adaptation regime as well as the representation. VulReaD's reported best-model variant on R2Vul raises binary F_1 from 0.64 to 0.77 and CWE macro-F_1 from 0.05 to 0.30 when graph knowledge is added (source Table 6) [67]. Its no-KG condition uses R2Vul prompting; the comparison combines graph-guided preference training and inference context. CWE labels extracted from explanations assess label agreement, rather than the faithfulness of every reasoning step.

Documentation comparisons show how the selected outcome changes the conclusion. On Python repositories with DeepSeek V3.2, RepoAgent and RepoDoc achieve API coverage of 91.28% and 63.15%, respectively, but Completeness@10 of 76.53% and 78.92% (source Table 3) [111]. The generation backbone is shared, while the complete systems differ. API mention coverage favors RepoAgent; judged question answerability favors RepoDoc. Neither metric alone captures the entire documentation objective.

Other comparisons examine the policy for obtaining evidence. LingmaAgent separately removes call-reference expansion and repository summaries; AgenticSZZ compares deterministic ranking on the full temporal graph with model-guided investigation; DomAgent compares static and agentic access to the same knowledge/case pools [61, 92, 106]. These comparisons link graph mechanisms to control decisions, although access, reasoning, training, and total budgets are not uniformly held fixed. They support specific exploration designs, not an unconditional advantage for additional agent steps.

6.4 Validation must reflect the intended outcome

A useful check must distinguish an acceptable task result from the failure that should trigger another action. In SysKG-UTF, steps-to-reproduce section accuracy on 380 sampled Firefox reports is 0.708

for SoapOperaTG, 0.824 without few-shot examples, and 0.976 for the full method (source Table 1) [95]. These values measure extraction into the graph; scenario feasibility and human exploratory testing are assessed separately. Better extraction alone does not establish that a generated scenario reveals a defect.

Terraform knowledge injection separates technical validity from intent satisfaction. On 458 prompts with GPT-4o, naive RAG and Graph RAG produce 322 and 368 technical passes, respectively, but only 240 and 268 joint technical/intent passes (source Table 13) [68]. All counts use the same 458-prompt denominator. Graph RAG therefore achieves 80.3% technical success and 58.5% joint success, showing how the acceptance definition changes the assessed outcome.

EntailLLM illustrates a related distinction between a supported fraction and the number of supported candidates. With chain-of-thought generation on standard binaries covering three CWE classes, supplying domain-derived labels raises pooled entailment from 82.9% to 98.5%, while entailed paths decrease from 772 to 604 (source Appendix Table 2) [66]. Both conditions use the domain reasoner, so the contrast measures generation context. The higher entailment fraction does not establish broader path coverage.

Agent workflows make this distinction operational. For Llama 3.3 on 152 MicroSS scenarios, cloud-RCA path validity (PA@3) is 0.61/0.73/0.78 for Straight-Shot/ReAct/Plan-and-Execute, while correct fault hypotheses (HA@3) are 0.29/0.31/0.29 (source Table 3) [87]. LingmaAgent's added static-review agent reduces resolution from 21.33% to 18.33% (source Table 4) [61]. Thus, more inspection or feedback need not improve the task result in these configurations. MPC-Coder instead uses verification reports to repair control code, and ReProAgent revises tests against both execution and issue correspondence [127, 129]. These mechanisms identify the acceptance targets to which feedback must be connected.

Human-facing outcomes add a further criterion. CODENS obtains high grounding scores on 11 industrial-project questions while developers request less verbose, more workflow-oriented explanations [41]. RepoDoc's API coverage and judged answerability favor different systems, and VulReaD's extracted CWE labels assess classification agreement [67, 111]. Evaluation must therefore retain the intended use of the explanation as well as its factual or label-level checks.

6.5 Repository experiments examine graph support in practice

We add exploratory system comparisons on repository questions (Task 1: 56 StackRepoQA and 44 SWE-QA instances [4, 81]) and issue localization (Task 2: 47 Loc-Bench and 53 SWE-bench instances [16, 38]). They examine whether useful graph access translates into better task outcomes in the selected configurations; they do not estimate the isolated causal effect of graph structure. Each task uses 100 frozen instances sampled without replacement from repositories below 500,000 main-language lines of code; together they cover 82 repositories and 159 commits. One result is retained per method–instance pair, with failures reported separately. All conditions use the served model identifier `deepseek-flash`. The main run compares source browsing through Claude Code with four graph tools and BM25/FAISS retrieval; a supplementary run adds one-pass source-context answering and mini-swe-agent.

More efficient access need not improve quality. On common valid instances, mini-swe-agent exceeds source-context answering by 20.96 QA points, 9.19 file-hit percentage points, and 20.21 function-hit percentage points (99/98/94 instances). This contrast combines adaptive access, multi-step reasoning, and different context exposure. Within the Claude Code run, graph tools provide no uniform quality gain (Table 2). CodeGraph instead reduces mean tool calls by 47.0%/42.5% and answer time by 12.8%/13.0% on Tasks 1/2. These are answer-stage observations, excluding construction and historical failed runs; they do not establish equal quality or lower lifecycle expenditure.

Table 2. Complementary task results: single-run descriptive means. QA sums five Judge ratings (1–20 each); localization reports Hit@1 (%). Denominators are 100/100/96 for QA/files/functions, except mini-swe-agent (99/98/94). Four instances have no applicable function truth. Blocks differ in harness and Judge budgets; they are not matched ablations.

Method	QA /100	File Hit@1	Function Hit@1
Supplementary baseline run			
Source-context LLM	72.75	85.00	65.62
mini-swe-agent	94.09	93.88	87.23
Claude Code run			
Source-browsing agent	95.62	95.00	85.42
Agent + Graphify	94.34	94.00	88.54
Agent + GitNexus	94.64	94.00	85.42
Agent + CodeGraph	94.62	93.00	86.46
Agent + Understand Anything	95.24	93.00	85.42
Agent + BM25/FAISS	94.40	93.00	84.38

Access does not ensure selection. Every graph-augmented session accesses its graph. In Task 2, some query result mentions a reference file in 99/100 GitNexus and 97/100 CodeGraph sessions, yet file Hit@1 is 94% and 93%. Mention does not establish prominence or correct use. With source browsing already at 95% file Hit@1 and 53/100 questions containing a reference-identifier clue, aggregate gains have limited headroom. Traces distinguish retrieval, ranking, and function-identity failures, connecting the task results to the survey's separation of intermediate evidence and engineering outcomes.

Relative to source browsing, Graphify's QA score is higher on 33 questions, tied on 17, and lower on 50; its function Hit@1 gain comes from five wins, two losses, and 89 ties. Thus, the observed improvement concerns a small number of localization cases and does not extend uniformly to answer quality. Single runs do not establish stability across generation or Judge repetitions, and aggregate summaries cannot supply that uncertainty.

These system comparisons retain integration effects: Graphify and GitNexus recover from failed Skill discovery, and several methods receive extraneous batch metadata. Source truncation and differing Judge budgets further limit attribution. The trace analysis separates access failures, candidate ranking, function-identity expression, and execution overhead. The mini-swe-agent run also has three iteration-limit failures.

6.6 Controlled comparisons define the scope of a result

Comparisons of graph interventions should hold raw artifacts constant while varying lexical, dense, graph, or hybrid access; retaining code while removing paths isolates a different intervention from removing graph tools. CrossCodeEval and RepoBench distinguish cross-file completion, retrieval, and the combined pipeline [23, 59]. Interface and presentation effects also require controls [115, 117]. Reporting graph quality, retrieved evidence, agent actions, and task outcomes locates the affected stage.

The task determines the outcome oracle. SWE-bench uses issue and regression tests; DS-1000 combines functional and surface-form constraints [38, 45]. Reference patch locations are localization targets, while supporting evidence can include unchanged callers or tests. Task-provided information also changes difficulty: Explore-Construct-Verify's 25 API-recommendation questions

name the existing API, whereas KPL acquires intent and APIMig validates migration [35, 43, 97]. Their scores cannot be treated as interchangeable measures of graph reasoning.

Evaluation of reuse must specify source revisions and the availability of every derived object. Reusing a graph across trials estimates variation conditional on that graph; rebuilding also captures extraction variation. Paired task results, repository-level aggregation, and construction/model repetitions address different uncertainty sources. Related reports and repeated use of a benchmark do not provide independent replication of a mechanism.

6.7 The cost of graph knowledge depends on reuse

For a workload with n tasks and m updates, lifecycle cost can be decomposed as

$$C_{\text{total}} = C_{\text{build}} + \sum_{j=1}^{m} C_{\text{update},j} + \sum_{i=1}^{n} (C_{\text{retrieve},i} + C_{\text{model},i} + C_{\text{validate},i}).$$

Each cost account uses a consistent unit, such as time, tokens, or expenditure, over a stated reuse horizon. Storage is reported separately as retained capacity. Repeated queries amortize construction; frequent changes add refresh costs.

Output requirements can reverse a cost comparison. On seven Android apps plus constructed cases, comprising 17 flow-report XML files and 23 policies, the privacy checker reports 383,808 tokens and 1,625 seconds for its pure-LLM baseline. Hybrid binary decisions use 25,100 tokens and 207 seconds; adding explanations increases these totals to 94,538 tokens and 2,551 seconds (source Table II) [62]. The hybrid retains a token advantage but loses its latency advantage when producing reports. Token use and elapsed time should therefore be measured separately for matched outputs.

Quality must accompany these costs. Across questions on 31 repositories, one per programming language, with Claude Opus 4.6, Codebase-Memory averages 2.3 tool calls and approximately 1,000 tokens per question versus 4.8 calls and approximately 10,000 tokens for file exploration. First-author judgments against manually derived references score the answers 0.83 and 0.92, respectively [102]. This setting exposes a cost–quality trade-off in structural lookup.

Larger action spaces can also increase task-time cost. CodexGraph reports higher token consumption with its graph-query interface, attributing part of the increase to query-result length and unsuccessful query cycles [60]. RepoGraph reports additional costs when agents repeatedly request graph context [71]. These observations complement the savings reported by Codebase-Memory: graph access changes how work is performed, and its cost depends on the resulting trajectory.

The requirements-retrieval study projects 51.8% lower inference energy by shortlisting candidate pairs under a fixed mean cost per inference [78]. The projection applies to graph and vector retrieval and excludes construction and infrastructure. It quantifies pruning during requirements analysis. Maintenance support incurs recurring refresh work, as in RepoDoc's selective documentation regeneration [111]. Both uses require build and update costs over their actual reuse horizons.

7 Challenges and Opportunities

7.1 Finding 1: Graph access serves different purposes before and after evidence is localized

DSrepair changes which API facts reach repair, whereas KGCompass holds localized candidates fixed and changes whether relationships among them are supplied [72, 115]. The former addresses missing content; the latter changes how available evidence supports a choice. SEER, DomAgent, and SGAgent introduce a further decision: whether observations justify another retrieval step [88, 106, 125]. Taken together, these studies distinguish fact acquisition, relational presentation, and evidence-sufficiency control. Once a relevant file is known, additional retrieval may add little,

while a dependency path or a decision to stop can still change the outcome. The open problem is to identify the decision that remains unresolved and select the graph operation that addresses it.

The difficulty begins with deciding what the graph should represent. Package dependencies in DepsRAG, development-artifact paths in KGCompass, and pointer relations in PtrTrans serve different reasoning needs [7, 115, 122]. A schema built for one task can omit relations needed for another. For example, a call graph may identify the callers affected by a signature change without explaining the compatibility requirement behind that change. Moreover, an absent edge can reflect incomplete extraction rather than an absent dependency. Expanding every neighborhood increases the evidence available, but also the amount the agent must inspect; the controller needs a way to detect missing information without treating graph completeness as given.

One direction is to construct and extend graphs around explicit task questions. An initial graph could cover the entities named in an issue, then add relation types when an unresolved symbol, conflicting requirement, or failed test reveals a missing dependency. Before expanding, the agent could consult the source artifact to distinguish an extraction failure from genuinely missing knowledge. The distinction between resolved code references, generated concepts, and extracted rationale in existing construction methods motivates retaining provenance for each added relation [21, 100, 102]. Such provenance could guide when to trust a relation, verify it against code, or request clarification. Comparing fixed broad, fixed narrow, and expandable schemas on tasks outside their initial assumptions would test whether adaptation improves coverage without introducing harmful edits.

Another direction is to make evidence acquisition responsive to intermediate observations. RANGER and RepoHyper provide alternative retrieval mechanisms, while SEER, DomAgent, and SGAgent illustrate decisions to obtain further context [83, 88, 91, 106, 125]. A controller could record unresolved questions and choose the next operation according to which question it can answer: traversal for dependencies, text search for unlinked evidence, or source inspection for an uncertain relation. Repeatedly retrieving the same facts should prompt a different operation or a stopping decision. Evaluation should distinguish tasks with missing facts from those with localized evidence, hold sources and budgets fixed, and measure accepted outcomes alongside redundant calls and premature stopping. Comparisons using the same retrieved facts with and without paths would further separate relational presentation from additional content; any policy-training benefit should be assessed separately from graph access.

7.2 Finding 2: Graph-supported validity can diverge from engineering acceptance

Terraform knowledge injection improves technical and joint technical/intent success under separate criteria, while the cloud-diagnosis comparison increases path validity without a corresponding improvement in fault-hypothesis accuracy [68, 87]. LingmaAgent's static-review variant further shows that adding a checking stage can reduce issue resolution in the tested configuration [61]. The shared distinction is the target of the check: valid infrastructure, a supported reasoning path, and an accepted engineering result are different outcomes. For an agent, treating an intermediate pass as task acceptance can terminate investigation too early; treating an unverified warning as a repair requirement can redirect an otherwise useful action. Shared errors are an additional design risk: when generation and checking use the same incomplete graph, their agreement can preserve the same missing condition.

Future work could connect graph-derived checks to independent evidence about task correctness. EntailLLM and KGIT provide logical checking mechanisms, while MPC-Coder and ReProAgent demonstrate revision using external checks [66, 104, 127, 129]. Building on these mechanisms, a controller could use dependency paths to select affected tests and attach each check to the requirement or property it addresses. Execution results, formal counterexamples, and issue correspondence

would then inform whether to accept a candidate or investigate further. Test selection also needs protection against graph omissions: evaluating only graph-selected tests could miss the very dependencies that were not extracted. Broader regression checks or independently selected tests could provide a complementary check on this boundary.

Handling disagreement is a separate research problem. If a patch passes graph-derived constraints but fails a relevant test, the agent needs to investigate the implementation and the adequacy of the represented constraints. If a graph warning conflicts with source code or the current API version, changing the patch to satisfy that warning may introduce an error. A useful controller could retain the evidence behind each check and distinguish failures that call for artifact revision, knowledge correction, or clarification of intent. Comparing these responses with unconditional regeneration would test whether diagnosing the disagreement reduces repeated unsuccessful edits. Experiments should hold candidates fixed when comparing graph-derived, external, and combined checks, and include missing relations, misleading relations, and behaviorally incorrect candidates. Independent outcome labels are needed to measure both false acceptance and false rejection.

Human review is particularly relevant when requirements or explanations have no complete executable oracle. SEER uses expert requirements review, while CODENS exposes developer needs beyond grounding metrics [41, 88]. Future interfaces could present the requirement at issue, the source supporting a relation, and the unresolved disagreement so that a developer can correct the assumption driving the agent's next action. Studies could compare source evidence alone with paths and provenance annotations, measuring review effort, correction accuracy, and recovery after misleading advice. The objective is to establish when graph support helps a reviewer reach an appropriate decision and when it merely makes an incorrect recommendation appear persuasive.

7.3 Finding 3: Graph refresh does not establish the freshness of dependent artifacts

The maintenance mechanisms operate on different objects. Codebase-Memory replaces changed source records, Prometheus retains retrieved context with links inside its memory store, TestAgent changes graph-linked tests, and RepoDoc propagates source edits into documentation [75, 102, 111, 126]. An update to the graph, an internally consistent context store, and a refreshed document therefore establish different guarantees. RepoDoc's commit-based evaluation and SemanticForge's refresh-cost and stale-graph comparisons provide evidence for particular maintenance operations; WebDesignIter and IntelliTest add partial longitudinal observations [79, 99, 111, 128]. Together they make the dependency between a retained object and its changing source central to the synthesis. These results do not establish that a graph update also invalidates every cached explanation, test plan, or downstream artifact used by an agent.

The challenge is to identify which earlier conclusions a change invalidates. After a function signature changes, the graph may contain the new declaration while an agent still holds a summary, test plan, or retrieved context based on the previous version. Concurrent agents can also consume different revisions while contributing to the same task. File-level freshness alone cannot establish whether their retained evidence remains applicable. Conversely, discarding all memory after every edit would lose useful context about unchanged components and design decisions. Maintenance therefore requires explicit dependencies between source revisions, graph relations, and the artifacts derived from them.

A promising direction is to make these dependencies available to both the knowledge service and the agent. Each retained item could record its source revision and derivation dependencies; a change could mark affected items as stale and require refresh before they support an edit. Graphiti MCP's temporal episodes and Code Digital Twin's version-scoped knowledge motivate investigating such coordination across sessions [82, 116]. Research should examine how to preserve identities through renames and file moves, handle branch-specific facts, and propagate invalidation to summaries

and plans. Immediate refresh, recomputation on demand, and discarding affected objects make different trade-offs. Signature changes, API upgrades, and reversions would provide concrete cases for comparing missed invalidations, unnecessary recomputation, and the time until dependent state becomes consistent.

Maintenance also determines whether long-term reuse is economical. Codebase-Memory exposes a cost–quality tradeoff, the privacy checker separates token and latency savings, and the complementary tasks measure answer-stage efficiency (Section 6.5) [62, 102]. The relevant workload property is the number of useful reuses before knowledge becomes stale. If coding agents accelerate edits, re-extraction and cache invalidation can erode per-query savings, although local changes may leave much of the graph reusable. Future systems could refresh frequently used, rapidly changing regions more often and retain stable knowledge for longer, while checking validity before reuse. This creates a research question about maintenance policy: which knowledge should be kept ready for the next task, and which can be reconstructed when needed?

Answering that question requires longitudinal workloads that vary reuse, change frequency and scope, repository size, and output requirements. Persistent graphs, graphs built on demand, and direct exploration should be compared at an agreed quality threshold, including construction, refresh, inference, and validation costs. Historical replay must restrict both source facts and derived summaries to what was available at the task cutoff; BugRadar's partitions, AgenticSZZ's history, and APIMig's version-specific relations illustrate the importance of temporal identities [43, 92, 110]. Releasing source snapshots, agent decisions, and update events would enable analysis of how stale evidence affects later tasks and when the benefits of reuse exceed its maintenance costs.

8 Limitations

Our literature coverage is bounded by the search sources, time window, and selection criteria. Keyword-guided screening may favor studies that explicitly use graph-related terminology or provide detailed descriptions of their mechanisms, while institutional full-text restrictions may introduce selection bias.

Interpretation is further complicated by inconsistent terminology and varying levels of methodological detail across studies of knowledge graphs, code graphs, GraphRAG, and software agents. Two reviewers independently assessed the reports, resolved disagreements through discussion, and applied operational definitions supported by page-linked evidence. Control categories distinguish model selection of an operation from prescribed feedback and developer-led continuation; broader definitions of agency could group these mechanisms differently. Insufficient disclosure is not evidence that adaptation is absent.

Finally, our findings depend on the specific study versions and evaluation settings examined. Inspected papers may differ from subsequent publications in their methods or results, although version-specific references and source hashes help preserve traceability. Differences in models, prompts, budgets, tasks, and validation procedures further limit direct comparisons and causal attribution across systems. We therefore distinguish reported mechanisms from effects supported by within-study comparisons and avoid generalizing beyond the available evidence.

9 Conclusion

We systematically reviewed 104 reports to examine graph-supported software engineering and its integration with agent workflows. Five intervention roles describe what graph knowledge changes, while a separate control analysis identifies who chooses subsequent operations. Source–Intervention–Validation–Update connects these mechanisms to the evidence needed to assess their outcomes. Across studies, acquiring missing facts differs from presenting relations among known candidates and deciding whether to continue exploration; graph-supported validity can diverge

from engineering acceptance; and refreshing source records does not establish freshness of every dependent artifact.

Reported comparisons support particular instances of these distinctions but often combine changes to context, retrieval, and workflow. Exploratory repository-task comparisons also show no uniform task-quality gain from graph augmentation in the examined configurations, alongside some reductions in answer-stage work. These observations motivate evaluating which graph operation improves a particular decision, whether its feedback supports the intended acceptance criterion, and how its retained evidence remains valid across revisions. Adaptive access and coordinated maintenance require further controlled and longitudinal evaluation.

References

[1] Samuel Abedu, SayedHassan Khatoonabadi, and Emad Shihab. 2026. Synergizing LLMs and Knowledge Graphs: A Novel Approach to Software Repository-Related Question Answering. *ACM Transactions on Software Engineering and Methodology* 35, 10 (sep 2026), 1–42. doi:10.1145/3796510

[2] Samuel Abedu, Laurine Menneron, SayedHassan Khatoonabadi, and Emad Shihab. 2025. RepoChat: An LLM-Powered Chatbot for GitHub Repository Question-Answering. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 255–259. doi:10.1109/msr66628.2025.00045

[3] Sajid Ahmed, Abdullah Soomro, Kishor Kumar, Ubaidullah alias Kashif, Muhammad Raees, and Wajahat Akbar. 2026. Knowledge Graphs for Software Engineering: A Systematic Review of Applications, Techniques, and Emerging LLM Integration. *Global Research Journal of Natural Science and Technology* 4, 1 (Jan. 2026). doi:10.53762/grjnst.04.01.31

[4] Yoseph Berhanu Alebachew, Hunter Leary, Swanand Vaishampayan, and Chris Brown. 2026. Beyond Code Snippets: Benchmarking LLMs on Repository-Level Question Answering. arXiv:2603.26567 [cs.SE] doi:10.1145/3803846.3807467

[5] Aaron Alexander, Toma Fujii, and Theo Perrin. 2026. Repository-Level Code Generation with Semantic Knowledge Graphs and Constraint-Guided Planning. *International Journal of Science, Engineering and Innovative Research* 6, 1 (2026), 119–131. https://ijseir.org/index.php/ijseir/article/view/58

[6] Daniel Alfasi, Tal Shapira, and Anat Bremler-Barr. 2024. VulnScopper: Unveiling Hidden Links Between Unseen Security Entities. In *Proceedings of the 3rd GNNet Workshop on Graph Neural Networking Workshop (CoNEXT '24)*. ACM, 33–40. doi:10.1145/3694811.3697819

[7] Mohannad Alhanahnah and Yazan Boshmaf. 2024. DepsRAG: Towards Agentic Reasoning and Planning for Software Dependency Management. Version 5 consulted. arXiv:2405.20455 [cs.SE] https://arxiv.org/abs/2405.20455v5

[8] Alfred Asare Amoah and Yan Liu. 2025. Explainable Retrieval and Graph Augmented Code Vulnerability Repair Recommendations. In *2025 IEEE International Conference on Big Data (BigData)*. IEEE, 3924–3933. doi:10.1109/bigdata66926.2025.11402509

[9] Sunil Anasuri, Guru Pramod Rusum, and Kiran Kumar Pappula. 2023. AI-Driven Software Design Patterns: Automation in System Architecture. *International Journal of Artificial Intelligence, Data Science, and Machine Learning* 4, 1 (mar 2023), 78–88. doi:10.63282/3050-9262.IJAIDSML-V4I1P109

[10] Mihir Athale and Vishal Vaddina. 2025. Knowledge Graph Based Repository-Level Code Generation. Inspected arXiv version 2505.14394v1. arXiv:2505.14394 [cs.AI] doi:10.1109/LLM4Code66737.2025.00026

[11] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D C, Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B. Ashok, and Shashank Shet. 2023. CodePlan: Repository-level Coding using LLMs and Planning. Version 1 consulted. arXiv:2309.12499 [cs.SE] https://arxiv.org/abs/2309.12499v1

[12] Makar A. Batuev, Sergey V. Dorokhov, Denis V. Pleshivtsev, Mujahed A. Saif, and Maxim A. Medvedev. 2026. Development of an LLM-Based Q&A Assistant for the Code of an Internal Design System. In *2026 IEEE Ural-Siberian Conference on Biomedical Engineering, Radioelectronics and Information Technology (USBEREIT)*. IEEE, 1–4. doi:10.1109/usbereit70063.2026.11581004

[13] Yuanchen Bei, Weizhi Zhang, Siwen Wang, Weizhi Chen, Sheng Zhou, Hao Chen, Yong Li, Jiajun Bu, Shirui Pan, Yizhou Yu, Irwin King, Fakhri Karray, and Philip S. Yu. 2025. Graphs Meet AI Agents: Taxonomy, Progress, and Future Opportunities. Version 3. arXiv:2506.18019 [cs.AI] doi:10.48550/arXiv.2506.18019

[14] Lili Bo, Yuting He, Xiaobing Sun, Wangjie Ji, and Xiaohan Wu. 2024. A Software Bug Fixing Approach Based on Knowledge-Enhanced Large Language Models. In *2024 IEEE 24th International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 169–179. doi:10.1109/qrs62785.2024.00026

[15] Xiayu Cao, Rui Kang, Jian Tang, Qi Zhang, Yefeng Li, and Ziwei Xue. 2026. PowerReviewer: A Knowledge-Graph Enhanced Multi-Agent Framework for Automated Code Review in Power Grid Systems. In *2026 9th International Conference on Advanced Algorithms and Control Engineering (ICAACE)*. IEEE, 2549–2555. doi:10.1109/icaace69793.2026.11508786

[16] Zhaoling Chen, Xiangru Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. 2025. LocAgent: Graph-Guided LLM Agents for Code Localization. arXiv:2503.09089 [cs.SE] https://arxiv.org/abs/2503.09089

[17] Zihan Chen, Lei Zheng, and Di Zhu. 2026. A Survey of Agentic GraphRAG: From Retrieval-augmented Generation to Graph-native Agents. SSRN preprint. Posted May 2026. doi:10.2139/ssrn.6713979

[18] Manideep Reddy Chinthareddy. 2026. Reliable Graph-RAG for Codebases: AST-Derived Graphs vs LLM-Extracted Knowledge Graphs. Inspected arXiv version 2601.08773v1. arXiv:2601.08773 [cs.SE] https://arxiv.org/abs/2601.08773v1

[19] Tianyu Cui, Ruowei Fu, Changchang Liu, Yuhe Ji, Wenwei Gu, Shenglin Zhang, Yongqian Sun, and Dan Pei. 2025. AetherLog: Log-based Root Cause Analysis by Integrating Large Language Models with Knowledge Graphs. In *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 49–60. doi:10.1109/issre66568.2025.00019

[20] Jingwen Deng, Jihao Huang, Zhen Hao Wong, Hao Liang, Quanqing Xu, Bin Cui, and Wentao Zhang. 2026. Data-Centric Perspectives on Agentic Retrieval-Augmented Generation: A Survey. In *Findings of the Association for Computational Linguistics: ACL 2026*, Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (Eds.). Association for Computational Linguistics, San Diego, California, United States, 1570–1588. doi:10.18653/v1/2026.findings-acl.78

[21] Mouna Dhaouadi, Bentley Oakes, and Michalis Famelis. 2025. Automated Extraction and Analysis of Developer's Rationale in Open Source Software. *Proceedings of the ACM on Software Engineering* 2, FSE (June 2025), 2548–2570. doi:10.1145/3729383

[22] Mouna Dhaouadi, Bentley James Oakes, and Michalis Famelis. 2023. Towards Understanding and Analyzing Rationale in Commit Messages Using a Knowledge Graph Approach. In *2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. IEEE, 622–630. doi:10.1109/models-c59198.2023.00101

[23] Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. 2023. CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion. arXiv:2310.11248 [cs.LG] https://arxiv.org/abs/2310.11248

[24] Xueying Du, Yiling Lou, Mingwei Liu, Xin Peng, and Tianyong Yang. 2023. KG4CraSolver: Recommending Crash Solutions via Knowledge Graph. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*. ACM, 1242–1254. doi:10.1145/3611643.3616317

[25] Tim Eichhorn, Luís Ferreira Pires, Gayane Sedrakyan, and Ruben ten Asbroek. 2026. *CLAIR: Generating On-Demand Low-Code Application Documentation Through Knowledge Graph and LLM-Based Multi-agent System Integration*. Springer Nature Switzerland, 296–312. doi:10.1007/978-3-032-16234-2_24

[26] Alireza Ezaz, Ghazal Khodabandeh, Majid Babaei, and Naser Ezzati-Jivan. 2026. TAAF: A Trace Abstraction and Analysis Framework Synergizing Knowledge Graphs and LLMs. Inspected arXiv version 2601.02632v1. arXiv:2601.02632 [cs.SE] doi:10.1145/3744916.3787832

[27] Ximing Fan, Yong Fang, Peng Jia, Yang Liu, Yijia Xu, Xi Peng, and Yuhao Zhou. 2026. CuFuzz: An API-Knowledge-Graph Coverage-Driven Fuzzing Framework for CUDA Libraries. *Proceedings of the ACM on Software Engineering* 3, FSE (jun 2026), 3699–3721. doi:10.1145/3808170

[28] Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. 1987. The program dependence graph and its use in optimization. *ACM Transactions on Programming Languages and Systems* 9, 3 (July 1987), 319–349. doi:10.1145/24039.24041

[29] Marlene Goncalves, Adelina Figueira, and Graciela Perera. 2026. Large Language Models for Enhanced Code Retrieval in Cryptography Education. In *2026 IEEE Engineering Education World Conference (EDUNINE)*. IEEE, 1–5. doi:10.1109/edunine62390.2026.11547049

[30] Jutao Guo, Jingtao Chen, Yiping Shen, and Dunbing Tang. 2026. LLM-Assisted Knowledge Graph Construction and Dual-Layer Reasoning for Defect Repair Planning: A Neo4j-Based Demo. In *Proceedings of the 2026 International Conference on Artificial Intelligence and Agents (ICAIAgent '26)*. ACM, 1–8. doi:10.1145/3811238.3811550

[31] Lili Han, Bo Ma, Yan Zhou, and QiangMin. 2025. KNSE-Code: A Knowledge-Aware, Safety-Enhanced, Rule-Driven Framework for Aviation Safety-Critical Software Code Generation. In *2025 IEEE 6th International Conference on Computer, Big Data, Artificial Intelligence (ICCBD+AI)*. IEEE, 1–6. doi:10.1109/iccbdai66607.2025.11388648

[32] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia D'amato, Gerard De Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. 2021. Knowledge Graphs. *Comput. Surveys* 54, 4 (July 2021), 1–37. doi:10.1145/3447772

[33] Yinghan Hou and Zongyou Yang. 2026. AI Agent for Reverse-Engineering Legacy Finite-Difference Code and Translating to Devito. Inspected arXiv version 2601.18381v2. arXiv:2601.18381 [cs.AI] https://arxiv.org/abs/2601.18381v2

[34] Tianyuan Hu and Tong Wang. 2026. JDQuery: Query-Driven Defect Localization for Java Source Code Based on Code Knowledge Graphs. *Electronics* 15, 17 (aug 2026), 3827. doi:10.3390/electronics15173827

[35] Qing Huang, Zhenyu Wan, Zhenchang Xing, Changjing Wang, Jieshan Chen, Xiwei Xu, and Qinghua Lu. 2023. Let's Chat to Find the APIs: Connecting Human, LLM and Knowledge Graph through AI Chain. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 471–483. doi:10.1109/ase56229.2023.00075

[36] Hamed Jelodar, Samita Bai, Mohammad Meymani, Parisa Hamedi, Roozbeh Razavi-Far, and Ali Ghorbani. 2026. Integrating Graphs, Large Language Models, and Agents: Reasoning and Retrieval. Version 2 consulted. arXiv:2604.15951 [cs.AI] https://arxiv.org/abs/2604.15951v2

[37] Ming Jiang, Xixin Chen, and Yao Lu. 2026. VulKG-RAG: Graph-Enhanced Retrieval for Hierarchical Vulnerability Detection. In *2026 29th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*. IEEE, 987–992. doi:10.1109/cscwd68734.2026.11582592

[38] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? Version 3 consulted. arXiv:2310.06770 [cs.CL] https://arxiv.org/abs/2310.06770v3

[39] Jiazhen Kang, Yuchen Lu, Chen Jiang, Jinrui Liu, Tianhao Zhang, Bo Jiang, Ningyuan Sun, Tongtong Wu, and Guilin Qi. 2026. KCoEvo: A Knowledge Graph Augmented Framework for Evolutionary Code Generation. Inspected arXiv version 2603.07581v1. arXiv:2603.07581 [cs.SE] https://arxiv.org/abs/2603.07581v1

[40] Rabimba Karanjai, Sam Blackshear, Lei Xu, and Weidong Shi. 2024. Generating Move Smart Contracts based on Concepts. Inspected arXiv version 2412.12513v1. arXiv:2412.12513 [cs.SE] https://arxiv.org/abs/2412.12513v1

[41] Abdelhak Kelious, Chyrine Tahri, and Eliot Bardet. 2026. CODENS: Transforming Code Changes into Living, Accessible, and Queryable Documentation. Inspected arXiv version 2607.18356v1. arXiv:2607.18356 [cs.SE] doi:10.1145/3820755.3832807

[42] Mehmet Oğuz Kocadere, Yusuf Emir Cömert, Sudenaz Yazıcı, Şamil Giray Karaçay, İbrahim Baran Yıldız, and Tuğba Gürgen Erdoğan. 2026. Repowise: A Hybrid RAG-Based Conversational System for Software Repository Analysis. In *2026 8th International Congress on Human-Computer Interaction, Optimization and Robotic Applications (ICHORA)*. IEEE, 1–8. doi:10.1109/ichora69329.2026.11537220

[43] Li Kuang, Qi Xie, HaiYang Yang, Yang Yang, Xiang Wei, HaoYue Kang, and YingJie Xia. 2025. APIMig: A Project-Level Cross-Multi-Version API Migration Framework Based on Evolution Knowledge Graph. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI-2025)*. International Joint Conferences on Artificial Intelligence Organization, 7455–7463. doi:10.24963/ijcai.2025/829

[44] Kabul Kurniawan, Elmar Kiesling, and Andreas Ekelhart. 2025. CyKG-RAG: Towards knowledge-graph enhanced retrieval augmented generation for cybersecurity. In *Proceedings of the International Workshop on Retrieval-Augmented Generation Enabled by Knowledge Graphs (RAGE-KG 2024) (CEUR Workshop Proceedings, Vol. 3950)*. CEUR-WS.org, 51–64. Workshop held in November 2024; proceedings published on 9 April 2025. https://ceur-ws.org/Vol-3950/paper1.pdf

[45] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Scott Wen tau Yih, Daniel Fried, Sida Wang, and Tao Yu. 2022. DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation. Version 1 consulted. arXiv:2211.11501 [cs.SE] https://arxiv.org/abs/2211.11501v1

[46] Márk Lajkó, Péter Sánta, Balázs Nagy, and László Vidács. 2025. Optimizing Developer Assignment with Language Model and Knowledge Graph. In *2025 8th International Conference on Information and Computer Technologies (ICICT)*. IEEE, 254–259. doi:10.1109/icict64582.2025.00046

[47] Amirhossein Layegh, Amir H. Payberah, and Mihhail Matskin. 2026. From Code to Execution: Multi-Agent GraphRAG for Automated Artifact Generation. In *Proceedings of the Sixth European Workshop on Machine Learning and Systems (EuroMLSys '26)*. ACM, 147–154. doi:10.1145/3805621.3807637

[48] Minh Le-Anh, Nam Le Hai, Quyen Tran, Anh Nguyen Hoang, Linh Ngo Van, Bach Le, and Nghi D. Q. Bui. 2026. Beyond Repository Boundaries: Cross-Repository Graph Retrieval for Code Generation. Inspected arXiv version 2609.09987v1. arXiv:2609.09987 [cs.SE] https://arxiv.org/abs/2609.09987v1

[49] Xiaoqi Li, Hailu Kuang, Wenkai Li, Zongwei Li, and Shipeng Ye. 2026. CKG-LLM: LLM-Assisted Detection of Smart Contract Access Control Vulnerabilities Based on Knowledge Graphs. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering (ICSE-NIER '26)*. ACM, 91–95. doi:10.1145/3786582.3786817

[50] Xiaoqi Li, Shipeng Ye, Wenkai Li, and Zongwei Li. 2026. SCPatcher: Automated Smart Contract Code Repair via Retrieval-Augmented Generation and Knowledge Graph. Inspected arXiv version 2604.00687v1. arXiv:2604.00687 [cs.SE] https://arxiv.org/abs/2604.00687v1

[51] Yefeng Li, Rui Kang, Jian Tang, Qi Zhang, Xiayu Cao, and Changju Hu. 2026. PowerRecall: A Knowledge Graph Enhanced Heterogeneous Multi-Path Retrieval Framework for Power Software Code Review. In *2026 International Conference on Electrical, Control and Information Technology (ECITech)*. IEEE, 738–742. doi:10.1109/ecitech69277.2026.11601893

[52] Yue Li, Bohan Liu, Ting Zhang, Zhiqi Wang, David Lo, Lanxin Yang, Jun Lyu, and He Zhang. 2025. A Knowledge Enhanced Large Language Model for Bug Localization. *Proceedings of the ACM on Software Engineering* 2, FSE (jun 2025), 1914–1936. doi:10.1145/3729356

[53] Ruichao Liang, Jing Chen, Xianglong Li, Huangpeng Gu, Yebo Feng, Yue Xue, Cong Wu, and Yang Liu. 2026. EvoPoC: Automated Exploit Synthesis for DeFi Smart Contracts via Hierarchical Knowledge Graphs. Inspected arXiv version 2605.02868v1. arXiv:2605.02868 [cs.CR] https://arxiv.org/abs/2605.02868v1

[54] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2025. Large Language Model-Based Agents for Software Engineering: A Survey. arXiv:2409.02977 [cs.SE] https://arxiv.org/abs/2409.02977

[55] Mingwei Liu, Zheng Pei, Yanlin Wang, Zihao Wang, Zikang Li, Enci Lin, Xin Peng, and Zibin Zheng. 2026. Knowledge-Graph-Driven Data Synthesis for Low-Resource Software Development: A HarmonyOS Case Study. *Proceedings of the ACM on Software Engineering* 3, FSE (jun 2026), 2835–2857. doi:10.1145/3808133

[56] Mingwei Liu, Yanjun Yang, Yiling Lou, Xin Peng, Zhong Zhou, Xueying Du, and Tianyong Yang. 2023. Recommending Analogical APIs via Knowledge Graph Embedding. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*. ACM, 1496–1508. doi:10.1145/3611643.3616305

[57] Mingwei Liu, Chengyuan Zhao, Xin Peng, Simin Yu, Haofen Wang, and Chaofeng Sha. 2023. Task-Oriented ML/DL Library Recommendation Based on a Knowledge Graph. *IEEE Transactions on Software Engineering* 49, 8 (aug 2023), 4081–4096. doi:10.1109/tse.2023.3285280

[58] Ruitong Liu, Yaxuan Xie, Zexu Dang, Jinyi Hao, Xiaowen Quan, Yongcai Xiao, and Chunlei Peng. 2025. Dynamic Vulnerability Knowledge Graph Construction via Multi-Source Data Fusion and Large Language Model Reasoning. *Electronics* 14, 12 (jun 2025), 2334. doi:10.3390/electronics14122334

[59] Tianyang Liu, Canwen Xu, and Julian McAuley. 2023. RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. arXiv:2306.03091 [cs.CL] https://arxiv.org/abs/2306.03091

[60] Xiangyan Liu, Bo Lan, Zhiyuan Hu, Yang Liu, Zhicheng Zhang, Fei Wang, Michael Shieh, and Wenmeng Zhou. 2024. CodexGraph: Bridging Large Language Models and Code Repositories via Code Graph Databases. Version 2 consulted. arXiv:2408.03910 [cs.SE] https://arxiv.org/abs/2408.03910v2

[61] Yingwei Ma, Qingping Yang, Rongyu Cao, Binhua Li, Fei Huang, and Yongbin Li. 2025. Alibaba LingmaAgent: Improving Automated Issue Resolution via Comprehensive Repository Exploration. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25)*. ACM, 238–249. doi:10.1145/3696630.3728549

[62] Zhenyu Mao, Xinxin Fan, Yifei Wang, Jacky Keung, and Jialong Li. 2025. Hybrid Privacy Policy-Code Consistency Check using Knowledge Graphs and LLMs. Inspected arXiv version 2505.11502v1. arXiv:2505.11502 [cs.CR] https://arxiv.org/abs/2505.11502v1

[63] Ryan Anthony Marinelli and Victor Strandmoe. 2025. Durinn: Scaling Vibe Coding Auditing. In *2025 IEEE International Conference on Big Data (BigData)*. IEEE, 8291–8293. doi:10.1109/bigdata66926.2025.11400849

[64] Youssef Matloob, Hanan A. Kamal, and Nemat Abdelkader. 2025. Unlocking Code Understanding: A Local-First Hybrid Knowledge Graph Approach. In *2025 International Conference on Computer and Applications (ICCA)*. IEEE, 1–6. doi:10.1109/icca66035.2025.11430945

[65] Vatsal Kishorbhai Mavani. 2026. Codebase Aware Generative Agents for the SDLC: Automating Documentation, Dependency Analysis and Test Generation. In *2026 IEEE 5th International Conference on AI in Cybersecurity (ICAIC)*. IEEE, 1–4. doi:10.1109/icaic67076.2026.11395666

[66] Kaustuv Mukherji, Jaikrishna Manojkumar Patil, Colton Payne, Paulo Shakarian, Dana Warmsley, Nigel Stepp, and Evelyn Kim. 2026. EntailLLM: Verifying LLM-Generated Vulnerability Discovery Paths with Domain Knowledge via Logic Programming. Inspected arXiv version 2608.01763v1. arXiv:2608.01763 [cs.CR] https://arxiv.org/abs/2608.01763v1

[67] Samal Mukhtar, Yinghua Yao, Zhu Sun, Mustafa Mustafa, Yew Soon Ong, and Youcheng Sun. 2026. VulReaD: Knowledge-Graph-guided Software Vulnerability Reasoning and Detection. Inspected arXiv version 2602.10787v1. arXiv:2602.10787 [cs.SE] https://arxiv.org/abs/2602.10787v1

[68] Roman Nekrasov, Stefano Fossati, Indika Kumara, Damian Andrew Tamburri, and Willem-Jan van den Heuvel. 2026. IaC Generation with LLMs: An Error Taxonomy and A Study on Configuration Knowledge Injection. *ACM Transactions on Software Engineering and Methodology* (may 2026). doi:10.1145/3817608

[69] Bowei Ning, Xuejun Zong, Lian Lian, Kan He, Yifei Sun, Yuxiang Lei, and Plamen Vasilev. 2026. Securing the Dark Matter: A Semantic-Enhanced Neuro-Symbolic Framework for Supply Chain Analysis of Opaque Industrial Software. Inspected arXiv version 2605.07737v1. arXiv:2605.07737 [cs.SE] https://arxiv.org/abs/2605.07737v1

[70] Chenyu Niu, Liu Xin, Min Li, Jiyuan Huang, and Xiaoqiang Hu. 2026. A Knowledge Graph-Based Approach for Assisted PLC Code Generation. In *Proceedings of the 5th International Conference on Computer, Artificial Intelligence*

and Control Engineering (CAICE 2026). ACM, 308–314. doi:10.1145/3804601.3804649

[71] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. 2025. RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph. arXiv:2410.14684 [cs.SE] https://arxiv.org/abs/2410.14684

[72] Shuyin Ouyang, Jie M. Zhang, Zeyu Sun, and Albert Merono Penuela. 2025. Knowledge-Enhanced Program Repair for Data Science Code. Version 1 consulted. arXiv:2502.09771 [cs.SE] https://arxiv.org/abs/2502.09771v1

[73] Aswin Dev S P, Jerit Joshy, Mohammed Farhan T M, Praneeth C F, and Dimple Elizabeth Baby. 2025. Analysing Code-Based Retrieval Augmented Generation Methods for Knowledge Retention. In *2025 4th International Conference on Advances in Computing, Communication, Embedded and Secure Systems (ACCESS)*. IEEE, 61–65. doi:10.1109/access65134.2025.11135582

[74] Matthew J Page, Joanne E McKenzie, Patrick M Bossuyt, Isabelle Boutron, Tammy C Hoffmann, Cynthia D Mulrow, Larissa Shamseer, Jennifer M Tetzlaff, Elie A Akl, Sue E Brennan, Roger Chou, Julie Glanville, Jeremy M Grimshaw, Asbjørn Hróbjartsson, Manoj M Lalu, Tianjing Li, Elizabeth W Loder, Evan Mayo-Wilson, Steve McDonald, Luke A McGuinness, Lesley A Stewart, James Thomas, Andrea C Tricco, Vivian A Welch, Penny Whiting, and David Moher. 2021. The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *BMJ* 372 (March 2021), n71. doi:10.1136/bmj.n71

[75] Yue Pan, Zimin Chen, Siyu Lu, Zhaoyang Chu, Xiang Li, Han Li, Yang Feng, Claire Le Goues, Federica Sarro, Martin Monperrus, and He Ye. 2026. Prometheus: Towards Long-Horizon Codebase Navigation for Repository-Level Problem Solving. Inspected arXiv version 2507.19942v2. arXiv:2507.19942 [cs.SE] https://arxiv.org/abs/2507.19942v2

[76] Mihir Panchal, Arnav Deo, Varad Prabhu, Prinkal Doshi, Chetashri Bhadane, and Pranit Bari. 2026. LEDGE : Leveraging dependency graphs for enhanced context aware documentation generation. *Automated Software Engineering* 33, 2 (feb 2026). doi:10.1007/s10515-026-00596-y

[77] Santhosh Kusuma Kumar Parimi. 2026. CCCE: A Continuous Code Calibration Engine for Autonomous Enterprise Codebase Maintenance via Knowledge Graph Traversal and Adaptive Decision Gating. Inspected arXiv version 2604.13102v1. arXiv:2604.13102 [cs.SE] https://arxiv.org/abs/2604.13102v1

[78] Shreyas Patil, Pragati Kumari, Novarun Deb, and Gouri Ginde. 2026. Towards Energy-aware Requirements Dependency Classification: Knowledge-Graph vs. Vector-Retrieval Augmented Inference with SLMs. Inspected arXiv version 2603.23954v1. arXiv:2603.23954 [cs.SE] https://arxiv.org/abs/2603.23954v1

[79] Zheng Pei, Mingwei Liu, Zhenxi Chen, Zihao Wang, and Yanlin Wang. 2026. WebDesignIter: Co-Evolving Design Knowledge for Repository-Level Front-End Code Generation. Inspected arXiv version 2607.10621v1. arXiv:2607.10621 [cs.SE] https://arxiv.org/abs/2607.10621v1

[80] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2025. Graph Retrieval-Augmented Generation: A Survey. *ACM Transactions on Information Systems* 44, 2 (Dec. 2025), 1–52. doi:10.1145/3777378

[81] Weihan Peng, Yuling Shi, Yuhang Wang, Xinyun Zhang, Beijun Shen, and Xiaodong Gu. 2026. SWE-QA: Can Language Models Answer Repository-level Code Questions?. In *Findings of the Association for Computational Linguistics: ACL 2026*, Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (Eds.). Association for Computational Linguistics, San Diego, California, United States, 8230–8245. doi:10.18653/v1/2026.findings-acl.402

[82] Xin Peng and Chong Wang. 2026. Code Digital Twin: A Knowledge Infrastructure for AI-Assisted Complex Software Development. Inspected arXiv version 2503.07967v4. arXiv:2503.07967 [cs.SE] https://arxiv.org/abs/2503.07967v4

[83] Huy N. Phan, Hoang N. Phan, Tien N. Nguyen, and Nghi D. Q. Bui. 2024. RepoHyper: Search-Expand-Refine on Semantic Graphs for Repository-Level Code Completion. Version 4 consulted. arXiv:2403.06095 [cs.SE] https://arxiv.org/abs/2403.06095v4

[84] Rafael Ramires, Ana Respício, and Ibéria Medeiros. 2024. KAVE: A Knowledge-Based Multi-Agent System for Web Vulnerability Detection. In *2024 IEEE International Conference on Web Services (ICWS)*. IEEE, 489–500. doi:10.1109/icws62655.2024.00070

[85] Rafael Ramires, Ana Respício, Ibéria Medeiros, and Mike Papadakis. 2025. KAVe: A Tool to Detect XSS and SQLi Vulnerabilities using a Multi-Agent System over a Multi-Layer Knowledge Graph. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25)*. ACM, 1158–1162. doi:10.1145/3696630.3728601

[86] Simar Singh Rayat, Susheela Dahiya, Rajesh Soma, Poonam Kainthura, and Naincy Saroha. 2026. A Novel Agentic AI/ML Framework for Autonomous Detection, Diagnosis, and Optimization of Performance Bugs. In *2026 International Conference on Emerging Research in Smart Electronics and Machine Informatics (ECMI)*. IEEE, 1–9. doi:10.1109/ecmi68341.2026.11602867

[87] Evelien Riddell, James Riddell, Gengyi Sun, Michał Antkiewicz, and Krzysztof Czarnecki. 2026. Stalled, Biased, and Confused: Uncovering Reasoning Failures in LLMs for Cloud-Based Root Cause Analysis. In *Proceedings of the 2026 IEEE/ACM Third International Conference on AI Foundation Models and Software Engineering (FORGE '26)*. ACM,

172–183. doi:10.1145/3793655.3793732

[88] Mandira Roy, Novarun Deb, Nabendu Chaki, and Agostino Cortesi. 2026. SEER: Sustainability Enhanced Engineering of Software Requirements. *ACM Transactions on Software Engineering and Methodology* (jul 2026). doi:10.1145/3830904

[89] Satrio Adi Rukmono and Michel R.V. Chaudron. 2023. Enabling Analysis and Reasoning on Software Systems through Knowledge Graph Representation. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, 120–124. doi:10.1109/msr59073.2023.00029

[90] Iman Saberi and Fatemeh Fard. 2025. Context-Augmented Code Generation Using Programming Knowledge Graphs. Inspected arXiv version 2410.18251v2. arXiv:2410.18251 [cs.SE] https://arxiv.org/abs/2410.18251v2

[91] Pratik Shah, Rajat Ghosh, Aryan Singhal, and Debojyoti Dutta. 2025. RANGER – Repository-Level Agent for Graph-Enhanced Retrieval. Version 1 consulted. arXiv:2509.25257 [cs.SE] https://arxiv.org/abs/2509.25257v1

[92] Yu Shi, Hao Li, Bram Adams, and Ahmed E. Hassan. 2026. AgenticSZZ: Temporal Knowledge Graph-Guided Agentic Bug-Inducing Commit Identification. Version 2 consulted. arXiv:2602.02934 [cs.SE] https://arxiv.org/abs/2602.02934v2

[93] Manuel Stöger, Mario Bernhart, and Thomas Grechenig. 2026. From Ontologies to Repository Intelligence: A Review of Knowledge Graphs for Mining Software Repositories. In *15th International Conference on Advanced Information Technologies and Applications (ICAITA 2026)*. Sydney, Australia, 29–46. doi:10.5121/csit.2026.1601103

[94] Yanqi Su, Zheming Han, Zhenchang Xing, Xiwei Xu, Liming Zhu, and Qinghua Lu. 2023. SoapOperaTG: A Tool for System Knowledge Graph Based Soap Opera Test Generation. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 51–54. doi:10.1109/icse-companion58688.2023.00023

[95] Yanqi Su, Dianshu Liao, Zhenchang Xing, Qing Huang, Mulong Xie, Qinghua Lu, and Xiwei Xu. 2024. Enhancing Exploratory Testing by Large Language Model and Knowledge Graph. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. ACM, 1–12. doi:10.1145/3597503.3639157

[96] Yanbang Sun, Qing Huang, Xiaoxue Ren, Zhenchang Xing, Xiaohong Li, and Junjie Wang. 2025. Explore-Construct-Filter: An Automated Framework for Rich and Reliable API Knowledge Graph Construction. Inspected arXiv version 2502.13412v1. arXiv:2502.13412 [cs.SE] https://arxiv.org/abs/2502.13412v1

[97] Yanbang Sun, Qing Huang, Zhenchang Xing, Xiaoxue Ren, Xiaohong Li, Junjie Wang, Huan Jin, and Zhiping Liu. 2026. Balancing Richness and Reliability: An Explore-Construct-Verify Framework for API Knowledge Graph Construction. *ACM Transactions on Software Engineering and Methodology* (aug 2026). doi:10.1145/3840287

[98] Gou Tan, Zilong He, Min Li, Pengfei Chen, Jieke Shi, Zhensu Sun, Ting Zhang, Danwen Chen, Lwin Khin Shar, Chuanfu Zhang, and David Lo. 2026. LIDL: LLM Integration Defect Localization via Knowledge Graph-Enhanced Multi-Agent Analysis. Inspected arXiv version 2601.05539v1. arXiv:2601.05539 [cs.SE] https://arxiv.org/abs/2601.05539v1

[99] Akshat Sukhdeo Tiwari, Satya Ganesh Nutan Dev C, Pratik Madhukar Patole, Goutham Ponnamreddy, Srinivas Chinthalapudi, and Durga Prasad Kattamanchi. 2025. IntelliTest: An Intelligent Framework for Agentic Functional Test Generation Using Multimodal Data and Domain Knowledge. In *2025 IEEE Future Networks World Forum (FNWF)*. IEEE, 1–6. doi:10.1109/fnwf66845.2025.11317278

[100] Niko Usai, Dario Montagnini, Kristian Ilianov Iliev, and Raffaele Camanzo. 2026. LogicLens: Leveraging Semantic Code Graph to explore Multi Repository large systems. Inspected arXiv version 2601.10773v1. arXiv:2601.10773 [cs.SE] https://arxiv.org/abs/2601.10773v1

[101] Vijayamahantesh, Ashwitha A, Naresh E, and Piyush Kumar Pareek. 2026. TestAgentX: Leveraging Large Language Models and Reinforcement Learning for Scalable and Adaptive Software Testing. In *2026 International Conference on Next-Gen Quantum and Advanced Computing: Algorithms, Security, and Beyond (NQComp)*. IEEE, 599–604. doi:10.1109/nqcomp68334.2026.11497649

[102] Martin Vogel, Falk Meyer-Eschenbach, Severin Kohler, Elias Grünewald, and Felix Balzer. 2026. Codebase-Memory: Tree-Sitter-Based Knowledge Graphs for LLM Code Exploration via MCP. Version 1 consulted. arXiv:2603.27277 [cs.SE] https://arxiv.org/abs/2603.27277v1

[103] Chong Wang, Xin Peng, Zhenchang Xing, Yue Zhang, Mingwei Liu, Rong Luo, and Xiujie Meng. 2023. XCoS: Explainable Code Search Based on Query Scoping and Knowledge Graph. *ACM Transactions on Software Engineering and Methodology* 32, 6 (sep 2023), 1–28. doi:10.1145/3593800

[104] Jun Wang, Yanhui Li, Zhifei Chen, Lin Chen, Xiaofang Zhang, and Yuming Zhou. 2024. Knowledge Graph Driven Inference Testing for Question Answering Software. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. ACM, 1–13. doi:10.1145/3597503.3639109

[105] Lu Wang, Chenhan Sun, Chongyang Zhang, Weikun Nie, and Kaiyuan Huang. 2023. Application of knowledge graph in software engineering field: A systematic literature review. *Information and Software Technology* 164 (dec 2023), 107327. doi:10.1016/j.infsof.2023.107327

[106] Shuai Wang, Dhasarathy Parthasarathy, Robert Feldt, and Yinan Yu. 2026. DomAgent: Leveraging Knowledge Graphs and Case-Based Reasoning for Domain-Specific Code Generation. Inspected arXiv version 2603.21430v2. arXiv:2603.21430 [cs.AI] https://arxiv.org/abs/2603.21430v2

[107] Ziheng Wang, Maike Li, and Chen Zhi. 2026. Knowledge-Guided Synthetic Bug Feedback for LLM-Based Unit Test Generation. Inspected arXiv version 2607.11573v1. arXiv:2607.11573 [cs.SE] https://arxiv.org/abs/2607.11573v1

[108] Xiaoyang Wei, Zhengdi Wang, and Shuangyuan Yang. 2023. An Automatic Generation and Verification Method of Software Requirements Specification. *Electronics* 12, 12 (jun 2023), 2734. doi:10.3390/electronics12122734

[109] Claes Wohlin. 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering (EASE '14)*. ACM, 1–10. doi:10.1145/2601248.2601268

[110] Xi Xiao, Renjie Xiao, Qing Li, Jianhui Lv, Shunyan Cui, and Qixu Liu. 2023. BugRadar: Bug localization by knowledge graph link prediction. *Information and Software Technology* 162 (oct 2023), 107274. doi:10.1016/j.infsof.2023.107274

[111] Dong Xu, Mingwei Liu, Xiwen Wang, Jianfeng Zhong, and Zibin Zheng. 2026. RepoDoc: A Knowledge Graph-Based Framework to Automatic Documentation Generation and Incremental Updates. Version 1 consulted. arXiv:2604.26523 [cs.SE] https://arxiv.org/abs/2604.26523v1

[112] Hanxiang Xu, Wei Ma, Ting Zhou, Yanjie Zhao, Kai Chen, Qiang Hu, Yang Liu, and Haoyu Wang. 2024. CK-GFuzzer: LLM-Based Fuzz Driver Generation Enhanced By Code Knowledge Graph. Version 3 consulted. arXiv:2411.11532 [cs.SE] https://arxiv.org/abs/2411.11532v3

[113] Xiangzhe Xu, Guangyu Shen, Zian Su, Siyuan Cheng, Hanxi Guo, Lu Yan, Xuan Chen, Jiasheng Jiang, Xiaolong Jin, Chengpeng Wang, Zhuo Zhang, and Xiangyu Zhang. 2025. ASTRA: Autonomous Spatial-Temporal Red-teaming for AI Software Assistants. Inspected arXiv version 2508.03936v1. arXiv:2508.03936 [cs.CR] https://arxiv.org/abs/2508.03936v1

[114] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. 2014. Modeling and Discovering Vulnerabilities with Code Property Graphs. In *2014 IEEE Symposium on Security and Privacy*. IEEE, 590–604. doi:10.1109/sp.2014.44

[115] Boyang Yang, Jiadong Ren, Shunfu Jin, Yang Liu, Feng Liu, Bach Le, and Haoye Tian. 2025. KGCompass: Knowledge Graph Enhanced Repository-Level Software Repair. Version 2 consulted. arXiv:2503.21710 [cs.SE] https://arxiv.org/abs/2503.21710v2

[116] Cheng-Fu Yang, Hui-Chen Tsai, Juan Lu, and Jian-Chiun Liou. 2026. Graphiti Model Context Protocol Server: Implementation and Performance Evaluation of a Knowledge-graph-based AI Code Intelligence Framework. *Sensors and Materials* 38, 6 (jun 2026), 3585. doi:10.18494/sam6058

[117] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. Version 3 consulted. arXiv:2405.15793 [cs.SE] https://arxiv.org/abs/2405.15793v3

[118] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. Version 3 consulted. arXiv:2210.03629 [cs.CL] https://arxiv.org/abs/2210.03629v3

[119] V S N Lokesh Yarramallu, Rahul Jogi Gangireddy, Leela Naresh Manne, and Veena G. 2025. HybridCode: A Dual-Database Framework for Intelligent Codebase Analysis and Article Generation. In *2025 5th Asian Conference on Innovation in Technology (ASIANCON)*. IEEE, 1–8. doi:10.1109/asiancon66527.2025.11281189

[120] Jiao Yin, Wei Hong, Hua Wang, Jinli Cao, Yuan Miao, and Yanchun Zhang. 2024. A Compact Vulnerability Knowledge Graph for Risk Assessment. *ACM Transactions on Knowledge Discovery from Data* 18, 8 (jul 2024), 1–17. doi:10.1145/3671005

[121] Dengyong Yu. 2026. Dual-layer Graph Retrieval-Augmented Large Language Models for Terraform Code Generation. In *2026 6th International Symposium on Computer Technology and Information Science (ISCTIS)*. IEEE, 438–443. doi:10.1109/isctis70043.2026.11572212

[122] Zhiqiang Yuan, Wenjun Mao, Zhuo Chen, Xiyue Shang, Chong Wang, Yiling Lou, and Xin Peng. 2026. Project-Level C-to-Rust Translation via Pointer Knowledge Graphs. *Proceedings of the ACM on Software Engineering* 3, FSE (June 2026), 3675–3698. doi:10.1145/3808169

[123] Sungju Yun, Sijune Hwang, Yeonjoon Lee, Kyungtae Kang, and Sungbin Park. 2026. CLEAR: Causal Context-Based Agentic Reasoning for Vulnerability Detection. Inspected arXiv version 2608.03134v1. arXiv:2608.03134 [cs.CR] https://arxiv.org/abs/2608.03134v1

[124] Chunxiang Zhang, Shaowei Qin, Hao Wu, and Lei Zhang. 2024. Cooperative Mashup Embedding Leveraging Knowledge Graph for Web API Recommendation. *IEEE Access* 12 (2024), 49708–49719. doi:10.1109/access.2024.3384487

[125] Quanjun Zhang, Chengyu Gao, Yu Han, Ye Shang, Chunrong Fang, Zhenyu Chen, and Liang Xiao. 2026. SGAgent: Suggestion-Guided LLM-Based Multi-Agent Framework for Repository-Level Software Repair. *ACM Transactions on Software Engineering and Methodology* (May 2026). doi:10.1145/3818617

[126] Quanjun Zhang, Ye Shang, Siqi Gu, Jianyi Zhou, Chunrong Fang, Zhenyu Chen, and Liang Xiao. 2026. Multi-Agent LLM Collaboration for Unit Test Generation via Human-Testing-Inspired Workflows. Inspected arXiv version 2607.09101v1. arXiv:2607.09101 [cs.SE] https://arxiv.org/abs/2607.09101v1

[127] Quanjun Zhang, Yi Zheng, Ye Shang, Weifeng Sun, Haichuan Hu, Chunrong Fang, Zhenyu Chen, and Liang Xiao. 2026. ReProAgent: Tool-Augmented Multi-Stage Agentic Generation of Bug Reproduction Tests from Issue Reports. Inspected arXiv version 2607.09123v1. arXiv:2607.09123 [cs.SE] https://arxiv.org/abs/2607.09123v1

[128] Wuyang Zhang, Chenkai Zhang, Zhen Luo, Jianming Ma, Wangming Yuan, Chuqiao Gu, and Chenwei Feng. 2025. SemanticForge: Repository-Level Code Generation through Semantic Knowledge Graphs and Constraint Satisfaction. Inspected arXiv version 2511.07584v1. arXiv:2511.07584 [cs.SE] https://arxiv.org/abs/2511.07584v1

[129] Yinggang Zhang, Weiyi Xia, Ben Zhao, Tongwen Yuan, and Xianchuan Yu. 2026. MPC-Coder: A Dual-Knowledge Enhanced Multi-Agent System with Closed-Loop Verification for PLC Code Generation. *Symmetry* 18, 2 (Jan. 2026), 248. doi:10.3390/sym18020248

[130] Yan Zheng, Yang Zhou, Yichen Wang, Ying Shang, and Yong Liu. 2026. Dual Knowledge Graph Retrieval-Augmented Smart Contract Vulnerability Detection. In *2026 IEEE 50th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 1–10. doi:10.1109/compsac69091.2026.00130

[131] Bo Zhou, Jiaqi Shi, Ying Wang, Li Li, Tsz On Li, Hai Yu, and Zhiliang Zhu. 2025. Porting Software Libraries to OpenHarmony: Transitioning from TypeScript or JavaScript to ArkTS. *Proceedings of the ACM on Software Engineering* 2, ISSTA (jun 2025), 1445–1466. doi:10.1145/3728941

[132] Zulun Zhu, Tiancheng Huang, Kai Wang, Junda Ye, Xinghe Chen, and Siqiang Luo. 2026. Graph-Based Approaches and Functionalities in Retrieval-Augmented Generation: A Comprehensive Survey. *Comput. Surveys* 58, 10 (April 2026), 1–38. doi:10.1145/3795880